

Duale Hochschule Baden-Württemberg Mannheim

Studienarbeit

Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III

Studiengang Informatik

Studienrichtung Angewandte Informatik

Verfasser:	Ertan Ertas und Christian Strerath
Matrikelnummer:	9390224, 4965013
Kurs:	TINF23AI2
Studiengangsleiter:	Prof. Dr. Holger Hofmann
Betreuer:	Prof. Dr. Karl Stroetmann
Bearbeitungszeitraum:	15.10.2025 - 14.04.2026

Erklärung

Wir versichern hiermit, dass wir unsere Studienarbeit mit dem Thema: „Übersetzung von Vorlesungsunterlagen in TypeScript – Theoretische Informatik III“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.

Mannheim, 6. April 2026

Ertan Ertas

Kaiserslautern, 6. April 2026

Christian Strerath

Kurzfassung (Abstract)

Auf Deutsch

In English

Inhaltsverzeichnis

1	Einleitung	1
1.1	Problemstellung	1
1.2	Zielsetzung	2
1.3	Methodik und Abgrenzung	3
2	Theoretische Grundlagen der Typisierung	5
2.1	Semantik und Funktion von Datentypen	5
2.2	Statische Typisierung mittels TypeScript	7
2.2.1	Herausforderungen einer naiven Code-Migration	9
2.2.2	Explizite Typannotationen als architektonisches Prinzip	13
2.2.3	Strukturelles Typing und Interface-Verträge	17
2.3	Fazit zur architektonischen Neuausrichtung	22
3	RecursiveSet: API und fachliche Modellierung	24
3.1	Mathematische Mengen und Wohlunterscheidbarkeit	24
3.2	Die Datenstruktur RecursiveSet	25
3.3	Strukturierte Elemente: Die Tuple-Klasse	25
3.4	Assoziative Daten: Die RecursiveMap	26
3.5	Der Freeze-Contract: Immutabilität sichern	27
3.6	Von der Mengenkompensation zu expliziten Operationen . . .	27
3.7	Funktionale Transformationen als Komprehensions-Ersatz . . .	27
3.7.1	Basis-Transformationen: map, filter und reduce	29
3.7.2	Komplexe Konstruktionen: Verschachtelte Iterationen auflösen mit flatMap	29
4	RecursiveSet: Interne Architektur und Implementierung	31
4.1	Hash-Dispatcher und Gleichheitsprüfung	31
4.2	Speicherlayout: Structure of Arrays (SoA)	34
4.3	Kollisionsauflösung via Open Addressing	35
4.4	Löschvorgang: Backshift Deletion und Dense-Swap	36
4.5	Kapazitätsverwaltung und Hashing-Metadaten	37
4.6	RecursiveSet: Architektonische Spezialisierung	38
4.7	Performance-Optimierung funktionaler Methoden	38
4.8	Fazit der internen Architektur	38
5	Vom Python-Werkzeug PLY zu Lezer	40
5.1	Grundlagen von Lezer	41
5.2	Grammatikdefinition in Lezer	43
5.3	Generierung des Parsers	44

5.4	Integration in die TypeScript-Implementierung	46
6	Literaturverzeichnis	49
A	Anhang	50
A.1	Verzeichnis der portierten Lehrnotebooks	50

Abbildungsverzeichnis

Tabellenverzeichnis

1	Übersicht der übersetzten Notebooks im Repository	50
---	---	----

List of Listings

1	Mengen in Python: Das erwartete Verhalten	2
2	Mengen in TypeScript: Das fehlerhafte Verhalten	2
3	Unterschiedliche Operationen erfordern unterschiedliche Typen . .	5
4	Dynamische Typisierung in Python	6
5	Dynamische Typisierung in JavaScript	6
6	Typfehler wird erst zur Laufzeit sichtbar	6
7	Eine einfache Funktion in JavaScript	7
8	Dieselbe Funktion in TypeScript mit expliziten Typen	7
9	Einfache Funktion in Python	8
10	Einfache Funktion in JavaScript	8
11	Einfache Funktion in TypeScript	9
12	Naive Übersetzung einer Funktion nach TypeScript	10
13	Typprüfung zur Laufzeit in Python	10
14	Typprüfung zur Laufzeit in JavaScript und TypeScript	10
15	Der Typ <code>any</code> hebt die Typprüfung aus	11
16	Explizite Umwandlung in Python	11
17	Implizite und explizite Umwandlung in JavaScript	12
18	Eine Type Assertion ist keine echte Umwandlung	12
19	Unpräzise und präzise formulierte Funktion in TypeScript	13
20	Grundlegende TypeScript-Typen	14
21	Ein möglicher fehlender Rückgabewert in TypeScript	14
22	Möglicher fehlender Rückgabewert in Python	15
23	Ein möglicher fehlender Wert muss vor der Nutzung geprüft werden	15
24	Explizite Behandlung eines möglichen <code>undefined</code> -Werts	16
25	Der Typ <code>unknown</code> erzwingt eine vorherige Prüfung	16
26	Einengung von <code>unknown</code> durch Typprüfung	16
27	Type Hints in Python	17
28	Python <code>Any</code> vs. TypeScript <code>unknown</code>	17
29	Ein Interface als Vertrag über benötigte Methoden	18
30	Eine Klasse erfüllt ein Interface über ihre Struktur	18
31	Ein Protocol in Python als statischer Vertrag	19
32	Feste Datenstruktur mit <code>TypedDict</code> in Python	19
33	Äquivalente Objektstruktur in TypeScript	20
34	Eine generische Funktion in TypeScript	20
35	Eine generische Funktion in Python	20
36	Generics mit strukturellen Schranken im Vergleich	21
37	Union Type aus Literal Types in TypeScript	21

38	Literal Types in Python	21
39	Type Narrowing durch Kontrollfluss und <code>typeof</code>	22
40	Klassenprüfung mit <code>instanceof</code>	22
41	Vollständige Fallunterscheidung mit <code>never</code>	23
42	Mengenverhalten in Python	24
43	Mengenverhalten in TypeScript	25
44	Roboter-Koordinaten: Tupel in Python vs. TypeScript	26
45	Dictionaries in Python vs. <code>RecursiveMap</code> in TypeScript	27
46	Python: Mengenkompensation mit Selektion und Transformation	27
47	TypeScript: Funktionale Pipeline mit <code>filterMap</code> und Lambdas . . .	28
48	Python: Verschachtelte Mengenkompensation	29
49	TypeScript: Direkte Generierung und Vereinigung durch <code>flatMap</code> .	30
50	Der vollständige Hash-Dispatcher der Bibliothek	32
51	Die Gleichheitsfunktion <code>areEqual</code>	34
52	Speicherarchitektur von <code>RecursiveMap</code>	35
53	Lookup-Algorithmus mit linearem Sondieren	36
54	Reparatur der Suchkette (Backshifting)	37
55	Ply-Grammatik in Python	41
56	Lezer-Grammatik in TypeScript	41
57	Umsetzung Lezer-Grammatik in TypeScript	44
58	Erstellung des Parsers in TypeScript	45
59	<code>ParseStmnt</code> Funktion in TypeScript	47

Glossar

Backshift Deletion Ein intelligenter Löschalgorithmus für Hash-Tabellen (oft in Kombination mit Robin Hood Hashing genutzt). Statt Tombstones zu setzen, rücken nachfolgende kollidierte Elemente in die entstandene Lücke auf, was die Suchketten verkürzt und die Tabelle sauber hält.. 36

CPU-Cache Ein extrem schneller, aber kleiner Zwischenspeicher direkt im Prozessor. Die CPU lädt Daten bevorzugt in zusammenhängenden Blöcken (Arrays) in den Cache. Verstreute Objekte zwingen die CPU, Daten langsam aus dem Hauptspeicher (RAM) zu holen (Cache Miss).. 35

Garbage Collection Eine automatische Speicherverwaltung (z.B. in JavaScript oder Java), die den Arbeitsspeicher nach nicht mehr benötigten Objekten durchsucht und diese löscht. Das Erzeugen vieler winziger Objekte zwingt die Garbage Collection zu häufigem "Aufräumen", was Programme stark ausbremst.. 31

Hash-Tabelle Eine extrem schnelle Datenstruktur, die Daten in einem Array speichert. Ein Hash-Wert berechnet dabei direkt den Array-Index (den Speicherort), wodurch Elemente in idealerweise konstanter Zeit ($O(1)$) gefunden werden können.. 31

Kollision (Hashing) Tritt in einer Hash-Tabelle auf, wenn zwei unterschiedliche Eingabedaten zufällig denselben Hash-Wert generieren und somit auf denselben Speicher-Slot zugewiesen werden. Die Datenstruktur muss Mechanismen wie Open Addressing besitzen, um dies aufzulösen.. 33

Linear Probing Die einfachste Form des Open Addressing. Ist der Ziel-Slot in der Tabelle belegt, wird einfach der unmittelbar nächste Slot (Index + 1) geprüft, bis ein leerer Platz gefunden wird.. 35

Load Factor (Auslastungsgrad) Das Verhältnis von gespeicherten Elementen zur Gesamtanzahl der verfügbaren Slots in einer Hash-Tabelle. Ein Load Factor von 0.75 bedeutet, dass die Tabelle bei 75% Füllung vergrößert werden muss, da sonst zu viele Kollisionen entstehen.. 37

Open Addressing Eine Methode zur Auflösung von Hash-Kollisionen. Alle Elemente werden direkt im Array der Hash-Tabelle gespeichert. Ist ein errechneter Platz belegt, wird nach einem festgelegten Muster (z.B. Linear Probing) ein anderer freier Platz gesucht.. 35

Overhead Der zusätzliche Rechen- oder Speicheraufwand, der durch die Verwaltung, Hilfsfunktionen oder Abstraktionsschichten einer Programmiersprache entsteht und nicht direkt zur Lösung des eigentlichen Problems beiträgt.. 38

Speicherfragmentierung Ein Zustand im Arbeitsspeicher, bei dem Daten stark verstreut in kleinen Lücken liegen statt in einem zusammenhängenden Block. Dies verschlechtert die Zugriffszeiten der CPU enorm.. 35

Tombstone (Grabstein) Ein spezieller Marker-Wert in Hash-Tabellen mit Open Addressing. Er markiert einen Slot als "gelöscht", damit Suchketten von Elementen, die wegen einer Kollision nach hinten verschoben wurden, beim Nachschlagen nicht abreißen.. 36

XOR (Exklusives Oder) Eine bitweise logische Operation. Sie liefert wahr (1), wenn genau eins der Bits 1 ist, und falsch (0), wenn beide gleich sind. Da XOR kommutativ ($A \oplus B = B \oplus A$) ist, eignet es sich hervorragend, um reihenfolgeunabhängige Hash-Werte (z.B. für Mengen) zu berechnen.. 38

1 Einleitung

Die Theoretische Informatik bildet ein wichtiges Fundament im Informatikstudium. Themen wie Automaten oder formale Sprachen können anfangs jedoch sehr abstrakt wirken. Um diese theoretischen Konzepte greifbarer zu machen, kommen in der Vorlesung „Theoretische Informatik III“ interaktive Notebooks zum Einsatz. Dort wird erklärender Text direkt mit ausführbarem Programmcode gemischt. Bislang basieren diese Notebooks auf der Programmiersprache Python. Studierende können so theoretische Algorithmen direkt ausprobieren, verändern und live nachvollziehen.

Diese Arbeit befasst sich mit der Übertragung dieser Lehrinhalte in die Programmiersprache TypeScript. Der Fokus liegt dabei auf einer Portierung, welche nicht nur die Syntax übersetzt, sondern die Konzepte der formalen Sprachen durch feste, verbindliche Datenstrukturen explizit modelliert.

1.1 Problemstellung

Ausgangspunkt dieser Arbeit sind interaktive Python-Notebooks, die Konzepte der theoretischen Informatik demonstrieren. Werden diese Lehrmaterialien in das TypeScript-Ökosystem übertragen, reicht es jedoch nicht aus, lediglich die Syntax der Sprache zu übersetzen. Es entstehen fundamentale technische Hürden, da beide Sprachen Daten auf unterschiedliche Weise verarbeiten. Folgende zwei Problemfelder stehen dabei im Fokus:

Zu späte Fehlererkennung bei verschachtelten Daten

Python ist eine dynamische Sprache. Das bedeutet, dass Variablen keine festen Typen haben und Fehler oft erst dann auffallen, wenn das Programm ausgeführt wird und abstürzt. Zwar gibt es Werkzeuge, um Typen in Python zu prüfen, in der interaktiven Umgebung eines Jupyter-Notebooks übersehen diese jedoch häufig Fehler.

In der Vorlesung arbeiten wir ständig mit tief verschachtelten Objekten – beispielsweise Datenstrukturen, bei denen eine Liste wieder viele weitere Objekte und Listen enthält. Fehlt einem dieser verschachtelten Elemente versehentlich eine wichtige Eigenschaft, merkt Python das erst, wenn der Code im Hintergrund genau dieses fehlerhafte Element aufruft. Wir benötigen stattdessen eine Sprache, die uns schon während des Tippens garantiert, dass unsere Datenstrukturen über alle Ebenen hinweg fehlerfrei und vollständig aufgebaut sind.

Das unerwartete Verhalten von Mengen (Sets)

Das gravierendste Hindernis bei der Übersetzung zeigt sich jedoch bei der Arbeit mit Mengen (Sets). In der theoretischen Informatik ist es unumgänglich, berechnete Zustände in Mengen zu speichern, um Duplikate zu vermeiden oder später zu prüfen, ob ein Zustand bereits erreicht wurde.

In Python funktioniert das exakt so, wie man es intuitiv erwartet. Fügen wir ein Tupel in eine Menge ein und fragen danach ab, ob ein Tupel mit denselben Zahlen enthalten ist, liefert Python das erwartete Ergebnis:

```
1  menge = set()
2  menge.add((1, 2))
3
4  # Die Abfrage liefert das erwartete Ergebnis
5  print((1, 2) in menge) # Ausgabe: True
```

Listing 1: Mengen in Python: Das erwartete Verhalten

Ein naiver Übersetzungsversuch nach TypeScript führt hier jedoch zu einem fundamentalen Bruch in der Logik. Nutzen wir die Standard-Datenstruktur Set in TypeScript auf die exakt gleiche Weise, erhalten wir plötzlich ein falsches Ergebnis:

```
1  const menge = new Set();
2  menge.add([1, 2]);
3
4  // TypeScript findet das Array überraschenderweise nicht
5  console.log(menge.has([1, 2])); // Ausgabe: false
```

Listing 2: Mengen in TypeScript: Das fehlerhafte Verhalten

Obwohl das Array `[1, 2]` im Code zuvor offensichtlich in die Menge eingefügt wurde, behauptet TypeScript, es sei nicht vorhanden. Dieser zunächst unerklärliche Unterschied führt bei einer direkten Portierung dazu, dass Logik-Abfragen fehlschlagen, Endlosschleifen entstehen und Algorithmen falsche Ergebnisse liefern. Die Ursachenforschung und Behebung dieses Verhaltens ist die zentrale technische Herausforderung dieser Arbeit.

1.2 Zielsetzung

Das übergeordnete Ziel dieser Arbeit ist es, die Lehrinhalte der Vorlesung „Theoretische Informatik III“ vollständig nach TypeScript zu übersetzen. Die neue Co-

debasis soll die bisherigen Python-Notebooks ersetzen und dabei die klaren Strukturvorgaben der neuen Sprache nutzen, um Fehler frühzeitig zu vermeiden.

Daraus leiten sich folgende konkrete Teilziele ab:

1. **Strukturierte Neuimplementierung:** Die Algorithmen der Vorlesung sollen in TypeScript portiert werden. Die dynamische, oft fehleranfälligere Natur von Python soll dabei durch verbindliche Datenstrukturen und Typvorgaben ersetzt werden.
2. **Lösung des Mengen-Problems:** Um die Algorithmen fehlerfrei auszuführen, muss das unerwartete Verhalten von TypeScripts Set analysiert und behoben werden. Ziel ist die Entwicklung einer maßgeschneiderten Datenstruktur, die sich exakt wie ein Python-Set verhält und verschachtelte Objekte verlässlich erkennt.
3. **Einheitliche Ausführungsumgebung:** Da die Notebooks lokal ausgeführt werden, muss eine standardisierte Umgebung geschaffen werden, die unabhängig vom Betriebssystem identisch funktioniert. Ein wichtiges technisches Teilziel ist es dabei, die Hintergrundprozesse der Notebooks so anzupassen, dass TypeScripts strenge Prüfregeln auch bei der interaktiven, schrittweisen Ausführung zuverlässig erzwungen werden. Der Einrichtungsaufwand für Studierende soll durch eine klare Schritt-für-Schritt-Anleitung minimal gehalten werden.

1.3 Methodik und Abgrenzung

Die methodische Durchführung dieser Arbeit folgt einem fokussierten Ansatz. Der Umfang der Portierung beschränkt sich auf die nummerierten Lehrnotebooks der Vorlesung sowie deren unmittelbare technische Abhängigkeiten. Übungsnotebooks, die primär dem unbetreuten Selbststudium dienen, sind explizit vom Umfang dieser Arbeit ausgeklammert. Die genaue Auswahl der zu übersetzenden Artefakte erfolgte in Abstimmung mit dem Betreuer und ist im Anhang (siehe Abschnitt A.1) tabellarisch aufgeführt.

Ein zentraler methodischer Aspekt betrifft die didaktische Aufbereitung: Die begleitenden Erklärtexpte der Notebooks wurden nicht isoliert übersetzt, sondern inhaltlich an die neuen Strukturen von TypeScript angepasst. Dabei wurde bewusst darauf verzichtet, die neuen Konzepte kontinuierlich mit der alten Python-Implementierung zu vergleichen. Die Notebooks sollen als eigenständiges Lehrmaterial funktionieren, das die theoretischen Konzepte nativ und in sich geschlossen vermittelt.

Auf technischer Ebene war die Minimierung von Systemabhängigkeiten ein wesentliches Entwurfsziel. Insbesondere für die grafische Visualisierung von Automaten und Graphen wurde darauf geachtet, keine komplexen lokalen Installationen von externer Software vorauszusetzen. Stattdessen kommen integrierbare, plattformunabhängige Lösungen zum Einsatz, um die Ausführung auf verschiedenen Betriebssystemen zu garantieren und den Einstieg für die Studierenden so barrierefrei wie möglich zu gestalten.

Zur Gewährleistung der Reproduzierbarkeit und für die freie Zugänglichkeit der Ergebnisse wird der gesamte entwickelte Quellcode in einem öffentlichen Repository bereitgestellt. Die Implementierung ist unter der URL <https://github.com/cstrerath/formal-languages-typescript> im Verzeichnis TypeScript abrufbar.

2 Theoretische Grundlagen der Typisierung

Beim Übergang von den interaktiven Python-Notebooks der Vorlesung zu einer robusten TypeScript-Implementierung ändert sich nicht nur die Syntax. Der fundamentalste Unterschied liegt im Umgang mit Datentypen. Die in Python erprobten Algorithmen wurden ursprünglich in einer hochdynamischen Umgebung entwickelt. Um diese Konzepte in eine zuverlässig prüfbare, modulare Architektur zu überführen, ist ein strengeres Typkonzept unerlässlich. Typisierung ist in diesem Projekt daher kein Randthema, sondern das tragende Fundament der gesamten Codebasis.

2.1 Semantik und Funktion von Datentypen

Im Arbeitsspeicher eines Computers bestehen alle Daten letztlich nur aus Bitfolgen, also aus Nullen und Einsen. Für den Prozessor ist eine solche Bitfolge zunächst bedeutungslos. Erst durch einen „Typ“ erhält sie eine fachliche Interpretation. Der Typ legt fest, ob ein Wert beispielsweise als Zahl, als Text, als Wahrheitswert oder als komplexe Datenstruktur behandelt wird.

Man kann einen Typ daher als eine Art Vertrag verstehen. Dieser Vertrag beschreibt die Form eines Wertes und definiert, welche Operationen auf ihm sinnvoll und erlaubt sind. Bei einer Zahl sind mathematische Berechnungen naheliegend, bei einem Text dagegen das Zusammenfügen von Zeichenketten.

```
1 zahl = 42
2 text = "42"
3 print(zahl + 1)      # sinnvoll: 43
4 print(text + "!")    # sinnvoll: "42!"
5 print(text + 1)      # nicht sinnvoll: Text und Zahl passen hier nicht zusammen
```

Listing 3: Unterschiedliche Operationen erfordern unterschiedliche Typen

Ob eine Operation zulässig ist, hängt demnach maßgeblich vom Typ ab. Genau hier setzen Typsysteme an: Sie strukturieren die Bedeutung von Daten und dienen als essenzieller Sicherheitsmechanismus für Entwicklerinnen und Entwickler.

Wenn ein Programm versucht, eine Operation auf einem ungeeigneten Wert auszuführen, liegt oft kein syntaktischer Fehler vor, sondern ein inhaltlicher Widerspruch. Ein Typsystem macht solche Widersprüche sichtbar, bevor sie zu schwer nachvollziehbaren Abstürzen im laufenden Programm führen. Für die Modellierung unserer Parser und Datenstrukturen bedeutet das, dass wir erlaubte Zustände bereits im Code präzise und unmissverständlich ausdrücken können.

Prinzipien der dynamischen Typisierung

Python und JavaScript sind dynamisch typisierte Sprachen. Das bedeutet, dass Variablen nicht dauerhaft an einen vorher festgelegten Typ gebunden sind. Der Typ ergibt sich stattdessen aus dem Wert, der zur Laufzeit gerade in der Variablen gespeichert ist. Eine Variable kann also nacheinander Zahlen, Texte oder komplexe Objekte aufnehmen.

```
1 value = 42
2 value = "Hallo"
3 value = [1, 2, 3]
4 print(value)
```

Listing 4: Dynamische Typisierung in Python

Dasselbe Grundprinzip findet sich auch in JavaScript:

```
1 let value = 42;
2 value = "Hallo";
3 value = [1, 2, 3];
4 console.log(value);
```

Listing 5: Dynamische Typisierung in JavaScript

Diese Flexibilität erleichtert exploratives Arbeiten und schnelles Prototyping. Gleichzeitig verschiebt sie einen Teil der Verantwortung auf die Entwickelnden, da sich oft erst im Moment der Ausführung zeigt, ob eine Operation zum aktuellen Wert passt.

```
1 value = [1, 2, 3]
2 print(value + 1)
```

Listing 6: Typfehler wird erst zur Laufzeit sichtbar

Obwohl der inhaltliche Fehler – die Addition von Liste und Ganzzahl – im Quelltext offensichtlich ist, bemerkt Python ihn erst bei der tatsächlichen Ausführung der Zeile. Das Programm startet normal und stürzt erst an der problematischen Stelle ab.

Genau das ist der Kern dynamischer Typisierung: Die Sprache trifft viele Entscheidungen erst zur Laufzeit. In größeren Projekten erschwert dies die Fehlererkennung erheblich. Pflanzte sich ein fehlerhafter Wert über mehrere Funktionsaufrufe hinweg fort, tritt der Absturz oft erst tief im Inneren eines Algorithmus auf.

2.2 Statische Typisierung mittels TypeScript

TypeScript ist keine völlig neue Sprache, sondern eine statische Erweiterung von JavaScript. JavaScript bildet die Laufzeitgrundlage, während TypeScript Typinformationen hinzufügt, die bereits vor der Ausführung vom Compiler geprüft werden. Ein TypeScript-Programm definiert demnach nicht nur die Laufzeitlogik, sondern dokumentiert auch verbindlich, welche Arten von Werten an bestimmten Stellen erwartet werden.

Ein einfaches Beispiel verdeutlicht den Unterschied. Zunächst die JavaScript-Variante:

```
1 function add(a, b) {  
2     return a + b;  
3 }  
4 console.log(add(3, 4));  
5 console.log(add("Hallo", "Welt"));
```

Listing 7: Eine einfache Funktion in JavaScript

Da keine Typen vorgegeben sind, akzeptiert die Funktion beliebige Eingaben. Mit Zahlen berechnet sie eine Summe, mit Zeichenketten verkettet sie Texte. Die Flexibilität ist hoch, aber die eigentliche Absicht der Funktion bleibt unklar.

TypeScript erlaubt es, diese Absicht direkt im Code festzuhalten:

```
1 function add(a: number, b: number): number {  
2     return a + b;  
3 }  
4 console.log(add(3, 4));  
5 console.log(add("Hallo", "Welt"));
```

Listing 8: Dieselbe Funktion in TypeScript mit expliziten Typen

Durch die expliziten Typannotationen (`number`) wird die Signatur eindeutig. Der Code enthält nun eine überprüfbare Typaussage. Ein fehlerhafter Aufruf mit zwei Zeichenketten widerspricht dieser Festlegung und wird vom Compiler beanstandet, bevor das Programm überhaupt ausgeführt wird.

Dabei nimmt der Typ `number` in TypeScript eine besondere Rolle ein. Im Gegensatz zu Sprachen wie C oder Java, die streng zwischen ganzen Zahlen (`int`) und Kommazahlen (`float` oder `double`) unterscheiden, fasst TypeScript alle numerischen Werte unter dem Typ `number` zusammen. Unter der Haube handelt es sich dabei (wie in JavaScript) stets um 64-Bit-Fließkommazahlen nach dem

IEEE-754-Standard. Für unsere `add`-Funktion bedeutet das, dass sie ohne weitere Anpassungen der Signatur sowohl ganze Zahlen wie `add(3, 4)` als auch Kommazahlen wie `add(3.14, 2.5)` verarbeiten kann.

Statische Typisierung zieht somit einen entscheidenden Prüfschritt vor: Viele Fehlerklassen fallen bereits während der Entwicklung auf, nicht erst zur Laufzeit. Genau diese Eigenschaft macht TypeScript für die Portierung der Lehr-Notebooks ideal. Die Algorithmen und komplexen Datenstrukturen der theoretischen Informatik lassen sich so verlässlich beschreiben, prüfen und konsistent weiterentwickeln.

Komparative Gegenüberstellung der Typsysteme

Ein konkreter Vergleich einer kleinen Hilfsfunktion in Python, JavaScript und TypeScript verdeutlicht die praktischen Auswirkungen der verschiedenen Typsysteme. Die Funktion soll aus einer numerischen Start-ID eine textuelle Zustandsbezeichnung erzeugen. Wenn bereits bei solchen simplen Bausteinen Unklarheiten über erlaubte Datentypen bestehen, vervielfachen sich diese Probleme in komplexen Datenstrukturen schnell.

In Python ist die folgende Implementierung vollkommen zulässig:

```
1 def state_name(start_id):  
2     return "State_" + str(start_id)  
3 print(state_name(5))  
4 print(state_name("X"))
```

Listing 9: Einfache Funktion in Python

Da eine formale Typangabe für den Parameter `start_id` fehlt, bleibt die fachliche Absicht unklar. Die Funktion akzeptiert sowohl Zahlen als auch Texte, da beide am Ende in eine Zeichenkette umgewandelt werden können.

Analog verhält es sich in JavaScript:

```
1 function stateName(startId) {  
2     return "State_" + startId;  
3 }  
4 console.log(stateName(5));  
5 console.log(stateName("X"));
```

Listing 10: Einfache Funktion in JavaScript

Auch diese Funktion arbeitet mit verschiedenen Eingabetypen, solange der Operator syntaktisch anwendbar ist. Das wirkt zunächst flexibel, lässt aber offen, ob `startId` tatsächlich ein Text sein darf oder ob zwingend eine numerische ID erwartet wird. Der Code allein beantwortet diese Frage nicht.

Die TypeScript-Version macht die Anforderungen hingegen durch explizite Typannotationen sichtbar:

```
1 function stateName(startId: number): string {  
2     return "State_" + startId;  
3 }  
4 console.log(stateName(5));  
5 console.log(stateName("X"));
```

Listing 11: Einfache Funktion in TypeScript

Der Parameter `startId` ist eindeutig als Zahl definiert, das Ergebnis als Text. Der zweite Aufruf mit einer Zeichenkette (`"X"`) widerspricht diesem Vertrag und wird vom Compiler abgewiesen. Die Typinformation existiert nicht mehr nur implizit im Kopf der Entwickelnden, sondern ist maschinenlesbar im Quelltext verankert. Das macht den Code lesbarer und verhindert, dass sich ungültige Werte durch das System fortpflanzen.

Als Rückgabebetyp wird hier explizit `string` gefordert. Dieser Typ repräsentiert in TypeScript jegliche Art von Textdaten. Ähnlich wie in Python sind Strings in TypeScript unveränderlich (immutable) und können wahlweise mit einfachen (`'...'`), doppelten (`"..."`) oder sogenannten Backticks (``...``) definiert werden. Letztere sind als „Template Literals“ besonders nützlich, da sie das direkte Einbetten von Variablen und Ausdrücken erlauben.

2.2.1 Herausforderungen einer naiven Code-Migration

Eine reine Portierung von Python-Code nach TypeScript führt nicht automatisch zu echter Typsicherheit. Da TypeScript als Obermenge von JavaScript konzipiert ist, lässt sich bestehender Code oft fast unverändert in eine `.ts`-Datei übernehmen. Das Ergebnis ist lauffähig, nutzt die Stärken des Typsystems jedoch nicht.

Gerade bei der Übernahme bestehender Lehr-Notebooks ist die Versuchung groß, Code schnell zu portieren und die Typisierung auf später zu verschieben. Das folgende Beispiel zeigt eine solche naive Übersetzung:

Diese Funktion kompiliert fehlerfrei, verarbeitet aber unterschiedslos Zahlen, Strings und Arrays. Die fachliche Absicht – eine numerische Zustands-ID zu verarbeiten – geht verloren. Eine solche Übersetzung ändert letztlich nur die Dateienennung. Inhaltlich bleibt es JavaScript-Code, der die Offenheit und Fehleranfälligkeit der dynamischen Ausgangswelt nahtlos übernimmt.

```

1 function stateName(startId) {
2     return "State_" + startId;
3 }
4 console.log(stateName(5));
5 console.log(stateName("X"));
6 console.log(stateName([1, 2, 3]));

```

Listing 12: Naive Übersetzung einer Funktion nach TypeScript

Dynamische Typprüfung zur Laufzeit

Auch in untypisierten oder dynamischen Umgebungen haben Daten zur Laufzeit einen konkreten Typ. Dieser lässt sich mit entsprechenden Werkzeugen auslesen. In Python geschieht dies typischerweise mit der Funktion `type()`, in JavaScript und TypeScript mit dem Operator `typeof`.

```

1 print(type(42))
2 print(type("Hallo"))
3 print(type([1, 2, 3]))

```

Listing 13: Typprüfung zur Laufzeit in Python

```

1 console.log(typeof 42);
2 console.log(typeof "Hallo");
3 console.log(typeof true);

```

Listing 14: Typprüfung zur Laufzeit in JavaScript und TypeScript

Diese Beobachtung ist fundamental: Selbst wenn ein Wert im Code nicht explizit typisiert wurde, kann zur Laufzeit geprüft werden, ob es sich um eine Zahl, einen Text oder ein Objekt handelt.

Implikationen fehlender Typisierung (`any`)

Werden Variablen in TypeScript nicht explizit getypt (wie `startId` im Beispiel der naiven Übersetzung), weist der Compiler ihnen (soweit sich kein anderer Type logisch erschließt) den Typ `any` zu. Dieser Typ nimmt eine Sonderrolle ein: Er beschreibt keine bestimmte Form oder erlaubte Operationen, sondern deaktiviert die Typprüfung für diesen Wert faktisch.

`any` signalisiert dem Compiler: „Lasse diese Stelle unkontrolliert passieren.“

Ein typsicheres Programm müsste unterscheiden, ob `value` ein Array, ein String oder eine Zahl ist. Bei `any` entfällt diese Unterscheidung. Der Compiler

```

1 let value: any = [1, 2, 3];
2 console.log(value.toUpperCase());
3 console.log(value * 2);
4 console.log(value.notExistingMethod());

```

Listing 15: Der Typ `any` hebt die Typprüfung aus

lässt beliebige Methodenaufrufe zu, obwohl keinesfalls gesichert ist, dass sie zum Laufzeitwert (hier einem Array) passen. Das Programm wird unweigerlich abstürzen.

Werden zentrale Programmteile mit `any` annotiert, verkommt TypeScript wieder zu einer Art JavaScript in `.ts`-Dateien. Das Ziel der Portierung – die präzise und überprüfbare Modellierung von Schnittstellen und Datenflüssen – wird durch `any` systematisch unterlaufen.

Gefahren der impliziten Typkonvertierung (Koerzision)

Ein weiteres Problem, das bei mangelhafter Typisierung in TypeScript zum Tragen kommt, ist die *Koerzision* (implizite Typumwandlung). Hierbei führt die Sprache selbstständig Typanpassungen durch, ohne dass Entwickelnde dies explizit anfordern.

In diesem Punkt unterscheiden sich Python und JavaScript fundamental. Python verlangt für viele Operationen explizite Umwandlungen:

```

1 values = [1, 2, 3]
2 text = str(values)
3 print(text)          # "[1, 2, 3]"
4 print(type(text))    # <class 'str'>

```

Listing 16: Explizite Umwandlung in Python

Ohne den Aufruf von `str()` findet keine Umwandlung des Arrays in einen Text statt.

JavaScript (und damit auch fehlerhaft typisiertes TypeScript) ist an dieser Stelle deutlich nachgiebiger und erzwingt oft stillschweigende Umwandlungen, wenn der Operator es zulässt:

Im Ausdruck `values + ""` wird das Array automatisch umgewandelt. Solche impliziten Operationen transportieren die fachliche Absicht schlecht und bergen hohes Potenzial für schwer vorhersehbares Verhalten.

Arbeitet man in TypeScript aus Bequemlichkeit mit `any`, öffnet man genau dieser JavaScript-typischen Semantik wieder Tür und Tor. Implizite Umwandlungen

```

1  const values = [1, 2, 3];
2  const implicitText = values + "";
3  const explicitText = String(values);
4  console.log(implicitText); // "1,2,3"
5  console.log(explicitText); // "1,2,3"

```

Listing 17: Implizite und explizite Umwandlung in JavaScript

können ungeprüft stattfinden, und die gewünschte Strenge des neuen Systems geht verloren.

Type Assertions als explizite Compiler-Anweisung

Neben `any` bietet TypeScript mit der Typbehauptung (Type Assertion) `as` einen weiteren Mechanismus, der das Typsystem aufweicht. Für Einsteiger wirkt `as` oft wie eine bequeme Typumwandlung. Das ist jedoch ein Trugschluss.

Während eine echte Umwandlung (wie `str(values)` in Python oder `String(values)` in JavaScript) den tatsächlichen Laufzeitwert verändert, modifiziert `as` lediglich die Sicht des Compilers auf diesen Wert. Der zugrunde liegende Wert bleibt zur Laufzeit völlig unangetastet.

```

1  let values: any = [1, 2, 3];
2  // Keine echte Laufzeit-Umwandlung:
3  const text = values as string;
4  // Der zugrunde liegende Wert ist zur Laufzeit weiterhin ein Array.
5  console.log(text.toUpperCase());

```

Listing 18: Eine Type Assertion ist keine echte Umwandlung

Der Compiler meldet hier keinen Fehler, da ihm durch `as string` signalisiert wurde, der Wert solle fortan als Zeichenkette behandelt werden. Zur Laufzeit ist `text` jedoch weiterhin ein Array. Der Aufruf der String-Methode `toUpperCase()` führt daher unweigerlich zum Programmabsturz. Das inhaltliche Problem wurde nicht gelöst, sondern lediglich vor der Typprüfung versteckt.

Wenn `any` den Type Checker weitgehend ausschaltet, dann dient `as` oft dazu, ihn an einer bestimmten Stelle gezielt zu blenden. Beides läuft dem Ziel entgegen, Datenstrukturen präzise und verlässlich zu modellieren.

Methodischer Verzicht auf Typ-Bypässe

Sowohl `any` als auch `as` sind keine neutralen Komfortfunktionen, sondern Mechanismen, die die disziplinierende Wirkung des Typsystems aushebeln.

`any` führt dazu, dass unklare Stellen wieder wie JavaScript-Code behandelt werden. `as` erzwingt die Kompilierung durch eine bloße Behauptung, ohne dass sich der Laufzeitwert an diesen Vertrag anpasst. Beides dient der schnellen Lösung, statt fachlich sauber zu arbeiten.

Für die Portierung der Notebooks war das unerwünscht. Das Ziel war, den Python-Code nach TypeScript und nicht nach JavaScript zu übersetzen. Daher wurde auf die Verwendung von `any` und `as` bei der Übersetzung verzichtet.

2.2.2 Explizite Typannotationen als architektonisches Prinzip

Der konstruktive Gegenentwurf zu Ausweichmechanismen wie `any` oder `as` ist die explizite Typannotation. Dabei wird direkt im Programmtext angegeben, welcher Typ an einer bestimmten Stelle (etwa für Variablen, Parameter oder Rückgabewerte) zwingend erwartet wird.

Ein einfaches Beispiel zeigt den Unterschied:

```
1 function stateNameLoose(startId) {  
2     return "State_" + startId;  
3 }  
4 function stateNameStrict(startId: number): string {  
5     return "State_" + startId;  
6 }
```

Listing 19: Unpräzise und präzise formulierte Funktion in TypeScript

Während die erste Funktion nur aus dem Funktionsrumpf erraten lässt, was übergeben werden soll, formuliert die zweite Funktion einen eindeutigen Vertrag. Die fachliche Absicht verlässt den Kopf der entwickelnden Person und wird zu einer maschinell prüfbaren Aussage. Gerade in einer größeren Codebasis reduziert dies den Interpretationsspielraum erheblich und bildet das Fundament für verlässliche Schnittstellen.

Atomare und strukturierte Basistypen

Diese expliziten Annotationen greifen auf die Basistypen von TypeScript zurück. Neben den bereits betrachteten Typen `number` (für alle numerischen Werte) und `string` (für Zeichenketten) ist insbesondere der Typ `boolean` zentral.

Der Typ `boolean` repräsentiert Wahrheitswerte und kann ausschließlich die Zustände `true` oder `false` annehmen. Er entspricht damit exakt dem `bool`-Typ aus Python (`True` / `False`) und wird primär für logische Auswertungen, Bedingungen und Flags verwendet.

Neben diesen atomaren Werten lassen sich auch Arrays (etwa `string[]`) sowie einfache Objekttypen mit festgelegten Eigenschaften beschreiben:


```

1 let stateLabel: string = "q0";
2 let stateId: number = 0;
3 let isAccepting: boolean = false;
4 let alphabet: string[] = ["a", "b", "c"];
5 let rectangle: { width: number; height: number } = {
6     width: 10,
7     height: 20
8 };

```

Listing 20: Grundlegende TypeScript-Typen

Typen beschreiben in TypeScript also nicht nur primitive Einzelwerte, sondern auch zusammengesetzte Formen. Bereits der Typ von `rectangle` definiert hier eine einfache Struktur mit zwei klar benannten und typisierten Bestandteilen. Genau solche Beschreibungen dienen als Grundlage für komplexere Datenstrukturen.

Modellierung fehlender Werte (`null` und `undefined`)

In realen Programmen tritt häufig der Fall auf, dass ein gesuchter Wert schlicht nicht vorhanden ist. TypeScript verwendet dafür die Typen `null` und `undefined`.

Dabei bedeutet `undefined` typischerweise, dass ein Wert noch nicht initialisiert oder nicht gefunden wurde. `null` wird hingegen meist genutzt, um die absichtliche Abwesenheit eines Wertes explizit zu markieren. Für ein präzises Typsystem ist es essenziell, die Möglichkeit eines solchen fehlenden Wertes ausdrücklich zu modellieren, anstatt einfach darauf zu hoffen, dass der aufrufende Code damit zurechtkommt.

Die Suche nach einer E-Mail-Adresse in einer Datenbank verdeutlicht das:

```

1 function findEmail(username: string): string | undefined {
2     if (username === "admin") {
3         return "admin@example.com";
4     }
5     return undefined;
6 }

```

Listing 21: Ein möglicher fehlender Rückgabewert in TypeScript

Da die Funktion nicht zwingend ein Ergebnis findet, reicht der Rückgabotyp `string` allein nicht aus. An dieser Stelle greift der Code kurz einem späteren Konzept vor: dem logischen „Oder“, dargestellt durch den senkrechten Strich (`|`). Dieser sogenannte Union Type erlaubt es, mehrere zulässige Typen zu kombinieren. Die Signatur `string | undefined` hält somit ausdrücklich fest, dass

entweder eine Zeichenkette oder eben kein nutzbarer Wert zurückkommt. Die Abwesenheit wird so zu einem festen, maschinenlesbaren Teil des Vertrages.

Python modelliert dieses Problem auf der Ebene der Laufzeitwerte ganz ähnlich. Dort wird ein potenziell fehlender Wert typischerweise durch `None` ausgedrückt:

```
1 def find_email(username):  
2     if username == "admin":  
3         return "admin@example.com"  
4     return None
```

Listing 22: Möglicher fehlender Rückgabewert in Python

Der Grundgedanke ist identisch: Ein Rückgabewert kann vorhanden sein oder fehlen. Entscheidend ist jedoch, wie streng diese Möglichkeit anschließend durch den Compiler eingefordert wird.

Strikte Nullwert-Prüfung (`strictNullChecks`)

Die bloße Existenz von `null` und `undefined` erzeugt noch keine Typsicherheit. Ob der Compiler diese Fälle als eigenständige Problemquellen ernst nimmt, steuert in TypeScript die Option `strictNullChecks`.

Ist diese Option deaktiviert, behandelt TypeScript fehlende Werte nachsichtig. Ein potenziell fehlender Wert kann dann ungeprüft an Stellen weitergereicht werden, die zwingend einen konkreten Wert erfordern. Ist `strictNullChecks` hingegen aktiviert, muss der Fehlerfall vor der Verwendung abgefangen werden.

Führt man das vorherige Beispiel fort, wird der Unterschied sofort sichtbar:

```
1 const email = findEmail("guest");  
2 console.log(email.toUpperCase());
```

Listing 23: Ein möglicher fehlender Wert muss vor der Nutzung geprüft werden

Diese Zeile ist hochgradig fehleranfällig. Liefert die Suche `undefined`, schlägt der Aufruf von `toUpperCase()` zur Laufzeit fehl (in JavaScript ein berüchtigter „Cannot read properties of undefined“-Fehler). Mit aktiviertem `strictNullChecks` macht der TypeScript-Compiler diesen Widerspruch schon während des Tippens sichtbar und verweigert die Übersetzung.

Die saubere Lösung besteht darin, den fehlenden Wert im Kontrollfluss explizit zu behandeln:

Einer der häufigsten Fehler in vielen Programmen besteht darin, blind davon auszugehen, dass ein Wert vorhanden ist. Werden solche Annahmen vom Typsystem stillschweigend durchgewunken, verlagert sich der Programmabsturz in

```

1  const email = findEmail("guest");
2  if (email !== undefined) {
3      console.log(email.toUpperCase());
4  }

```

Listing 24: Explizite Behandlung eines möglichen undefined-Werts

die Laufzeit. Für die übersetzten Notebooks war `strictNullChecks` deshalb keine kosmetische Zusatzregel, sondern eine elementare Sicherheitsmaßnahme. Sie zwingt dazu, fehlende Werte nicht als Ausnahme, sondern als dokumentierten und überprüften Programmzustand zu begreifen.

Der Typ `unknown` als sichere Abstraktion

Neben `null` und `undefined` ist `unknown` ein weiterer wichtiger Spezialfall. Er ist besonders dann nützlich, wenn zwar feststeht, dass ein Wert übergeben wird, dessen genauer Typ aber noch nicht bekannt ist.

Auf den ersten Blick erinnert `unknown` an `any`, verfolgt jedoch das gegenteilige Ziel: Während `any` die Typprüfung faktisch deaktiviert, zwingt `unknown` den Entwickler dazu, den Wert vor der Verwendung präzise zu prüfen.

```

1  let value: unknown = [1, 2, 3];
2
3  // Nicht erlaubt, solange der Typ noch unbekannt ist:
4  console.log(value.toUpperCase());

```

Listing 25: Der Typ `unknown` erzwingt eine vorherige Prüfung

Der Compiler behandelt den Wert nicht als beliebig verwendbar, sondern verlangt ein sogenanntes *Type Narrowing* (Typ-Einengung). Erst nach einer erfolgreichen Typprüfung darf auf typspezifische Methoden zugegriffen werden:

```

1  let value: unknown = "State_1";
2
3  if (typeof value === "string") {
4      console.log(value.toUpperCase());
5  }

```

Listing 26: Einengung von `unknown` durch Typprüfung

So lassen sich unsichere oder externe Eingaben sicher modellieren, ohne den Type Checker auszuhebeln. Der Übergang vom Unbekannten zum Bekannten wird explizit und nachvollziehbar im Code festgehalten.

Vergleich mit statischer Analyse in Python (mypy)

Auf den ersten Blick besitzen Python und TypeScript ähnliche Ausdrucksmittel für Typen. In beiden Sprachen lassen sich Variablen, Parameter und Rückgabewerte annotieren.

```
1 def state_name(start_id: int) -> str:
2     return f"State_{start_id}"
```

Listing 27: Type Hints in Python

Ein fundamentaler Unterschied besteht jedoch in der Durchsetzung: Python ignoriert diese Type Hints zur Laufzeit vollständig. Ein fehlerhafter Aufruf mit falschen Datentypen führt nicht zwingend zu einem sofortigen Fehler. Um die Typinformationen systematisch zu prüfen, wird ein externes Werkzeug wie *mypy* benötigt, das den Code statisch analysiert.

Zudem fehlt Python ein strenges Äquivalent zu `unknown`. Der Typ `Any` in Python entspricht vielmehr dem unsicheren `any` in TypeScript: Er schaltet die Typprüfung für den jeweiligen Wert ab und erlaubt beliebige Operationen.

```
1 from typing import Any
2
3 def normalize(value: Any) -> str:
4     return str(value) # mypy erlaubt beliebige Eingaben
```

```
1 function normalize(value: unknown): string {
2     if (typeof value === "string") {
3         return value;
4     }
5     return String(value); // Zwang zur Prüfung oder Konvertierung
6 }
```

Listing 28: Python Any vs. TypeScript unknown

Während Python mit Type Hints und *mypy* eine mächtige, aber nachträgliche Typprüfung bietet, integriert TypeScript die statische Beschreibung und Kontrolle direkt in den regulären Entwicklungsprozess.

2.2.3 Strukturelles Typing und Interface-Verträge

Neben primitiven Werten und Funktionen erfordert die Modellierung von Architekturen verlässliche Verträge zwischen Komponenten. Ein solcher Vertrag legt

fest, welche Eigenschaften oder Methoden ein Wert besitzen muss, um in einem bestimmten Kontext genutzt werden zu können. TypeScript verwendet dafür unter anderem *Interfaces*.

Entscheidend ist dabei das Prinzip des *strukturellen Typings* (häufig als statisches Duck-Typing bezeichnet). TypeScript prüft nicht primär, ob ein Objekt aus einer spezifischen Klasse instanziiert wurde, sondern ob es die geforderte „Gestalt“ (Struktur) aufweist.

```
1 interface Structural {
2     equals(other: unknown): boolean;
3     hashCode(): number;
4     toString(): string;
5 }
```

Listing 29: Ein Interface als Vertrag über benötigte Methoden

Dieses Interface fordert lediglich das Vorhandensein dreier Methoden. Jede Klasse oder jedes Objekt, das diese Signatur erfüllt, ist typkompatibel:

```
1 class StatePair {
2     constructor(public left: number, public right: number) {}
3
4     equals(other: unknown): boolean {
5         return other instanceof StatePair
6             && this.left === other.left
7             && this.right === other.right;
8     }
9
10    hashCode(): number {
11        return this.left * 31 + this.right;
12    }
13
14    toString(): string {
15        return `${this.left}, ${this.right}`;
16    }
17 }
18
19 // Zuweisung ist zulässig, da die Struktur übereinstimmt:
20 const pair: Structural = new StatePair(1, 2);
```

Listing 30: Eine Klasse erfüllt ein Interface über ihre Struktur

Dieser strukturelle Ansatz entkoppelt Funktionalität von starren Klassenhierarchien. Datenstrukturen können polymorph mit völlig unterschiedlichen Elementen

arbeiten, solange diese den vertraglich zugesicherten Methodenapparat bereitstellen.

Äquivalente Strukturierung in Python (Protocol und TypedDict)

Python unterstützt strukturelles Denken zur Laufzeit nativ durch sein dynamisches Duck-Typing. Um dieses Verhalten jedoch statisch überprüfbar zu machen, bietet das `typing`-Modul (in Kombination mit `mypy`) die Klasse `Protocol`.

Ein `Protocol` definiert, analog zu einem TypeScript-Interface, welche Methoden ein Objekt bereitstellen muss:

```
1 from typing import Protocol
2
3 class Structural(Protocol):
4     def equals(self, other: object) -> bool: ...
5     def hash_code(self) -> int: ...
6     def __str__(self) -> str: ...
```

Listing 31: Ein Protocol in Python als statischer Vertrag

Klassen müssen in Python nicht explizit von diesem Protocol erben, um als kompatibel zu gelten; die Erfüllung der Signatur reicht aus. Während strukturelle Kompatibilität in TypeScript jedoch das Fundament des Typsystems bildet, ist sie in Python ein optionales, werkzeuggestütztes Konstrukt.

Für reine Datenobjekte (ohne Verhalten) bietet Python zudem `TypedDict`. Damit lassen sich Dictionary-Strukturen mit festen Schlüsseln und Typen definieren:

```
1 from typing import TypedDict
2
3 class Transition(TypedDict):
4     from_state: int
5     symbol: str
6     to_state: int
```

Listing 32: Feste Datenstruktur mit TypedDict in Python

In TypeScript übernehmen Interfaces oder Type-Aliase fließend beide Rollen, sie können sowohl Methodensignaturen als auch reine Datenfelder beschreiben:

Generische Programmierung im Sprachvergleich

Um Algorithmen und Datenstrukturen typsicher, aber dennoch wiederverwendbar zu gestalten, kommen *Generics* zum Einsatz. Sie dienen als Platzhalter für Typen, die erst bei der tatsächlichen Verwendung definiert werden, ohne auf `any` zurückgreifen zu müssen.

```

1 interface Transition {
2     from: number;
3     symbol: string;
4     to: number;
5 }

```

Listing 33: Äquivalente Objektstruktur in TypeScript

```

1 function first<T>(items: T[]): T | undefined {
2     if (items.length === 0) return undefined;
3     return items[0];
4 }
5
6 const firstNumber = first<number>([10, 20, 30]); // Typ: number / undefined
7 const firstLabel = first<string>(["q0", "q1"]); // Typ: string / undefined

```

Listing 34: Eine generische Funktion in TypeScript

Dasselbe Konzept wird in Python mittels `TypeVar` umgesetzt:

```

1 from typing import TypeVar
2
3 T = TypeVar("T")
4
5 def first(items: list[T]) -> T | None:
6     if len(items) == 0:
7         return None
8     return items[0]

```

Listing 35: Eine generische Funktion in Python

Die wahre Stärke von Generics entfaltet sich in der Kombination mit strukturellen Schranken. Mit dem Schlüsselwort `extends` (TypeScript) oder dem Parameter `bound` (Python) lässt sich erzwingen, dass der eingesetzte Typ bestimmte Verträge erfüllen muss:

Generics ermöglichen somit den Aufbau robuster, polymorpher Datenstrukturen, deren statische Typsicherheit vollständig erhalten bleibt.

Eingrenzung von Wertebereichen durch Literal und Union Types

Oft reicht es nicht aus, einen Wert pauschal als `string` oder `number` zu deklarieren. Wenn nur eine feste Menge spezifischer Werte fachlich zulässig ist, kommen *Literal Types* zum Einsatz. Ein Literal Type repräsentiert genau einen konkreten Wert (z. B. `"strict"`).

```

1 function containsStructural<T extends Structural>(items: T[], target: T): boolean {
2     return items.some(item => item.equals(target));
3 }

```

```

1 S = TypeVar("S", bound=Structural)
2
3 def contains_structural(items: list[S], target: S) -> bool:
4     return any(item.equals(target) for item in items)

```

Listing 36: Generics mit strukturellen Schranken im Vergleich

Ihre Flexibilität gewinnen Literal Types durch die Kombination mit *Union Types* (`|`), die ein logisches „Oder“ ausdrücken:

```

1 type Mode = "strict" | "loose";
2
3 let currentMode: Mode = "strict";
4 currentMode = "loose";
5 // currentMode = "debug"; // Compiler-Fehler: Ungültiger Wert

```

Listing 37: Union Type aus Literal Types in TypeScript

Dieses Konzept grenzt den Zustandsraum stark ein und transformiert Konventionen in maschinenprüfbare Verträge. Python unterstützt dies ebenfalls, nutzt dafür jedoch `Literal` aus dem `typing`-Modul:

```

1 from typing import Literal
2
3 Mode = Literal["strict", "loose"]
4
5 def run(mode: Mode) -> str:
6     if mode == "strict": return "Strenge Prüfung"
7     return "Lockere Prüfung"

```

Listing 38: Literal Types in Python

Kontrollflussbasierte Typverfeinerung (Type Narrowing)

Sobald ein Wert als Union Type (z. B. `string | number`) definiert ist, verlangt der Compiler vor typspezifischen Operationen Klarheit darüber, welcher Fall zur Laufzeit konkret vorliegt. Dieser Prozess der schrittweisen Typeinengung wird *Type Narrowing* genannt.

TypeScript analysiert dabei nicht nur einzelne Prüfoperatoren wie `typeof` oder `instanceof`, sondern wertet den gesamten Kontrollfluss (Verzweigungen, vorzeitige Abbrüche durch `return`) der Funktion aus.

```
1 function formatValue(value: string | number): string {
2     if (typeof value === "string") {
3         return value.toUpperCase(); // value ist hier vom Typ string
4     }
5     return value.toFixed(2); // value muss hier vom Typ number sein
6 }
```

Listing 39: Type Narrowing durch Kontrollfluss und `typeof`

Während `typeof` für primitive Datentypen geeignet ist, liefert es bei Objekten und Arrays pauschal `"object"`. Für die Überprüfung von Klasseninstanzen bietet sich daher `instanceof` an (analog zu `isinstance` in Python):

```
1 function describe(value: StatePair | string): string {
2     if (value instanceof StatePair) {
3         return `Paar: (${value.left}, ${value.right})`;
4     }
5     return value.toUpperCase();
6 }
```

Listing 40: Klassenprüfung mit `instanceof`

Besonders mächtig ist das Narrowing in Kombination mit Literal Types. Durch einfache Wertvergleiche kann der Compiler in komplexen `switch`- oder `if`-Konstrukten exakt ableiten, welche Typ-Zweige noch relevant sind.

Vollständigkeitsprüfung mittels `never`-Typ

Das Type Narrowing wirft die Frage auf, welcher Typ resultiert, wenn alle theoretisch möglichen Fälle einer Union vollständig abgehandelt wurden. In TypeScript nimmt dieser logisch un erreichbare Zustand den Typ `never` an.

`never` wird in der Praxis eingesetzt, um das *Exhaustiveness Checking* (Vollständigkeitsprüfung) zu erzwingen. Dies garantiert, dass Erweiterungen des Datenmodells (z. B. ein neues Token im Parser) im Kontrollfluss nicht übersehen werden:

2.3 Fazit zur architektonischen Neuausrichtung

Die Herausforderung der bestehenden Architektur lag nicht in einer grundsätzlichen Unzulänglichkeit von *mypy*. Für klassische, dateibasierte Python-Projekte ist

```

1  type TokenKind = "char" | "union" | "star";
2
3  function describeToken(kind: TokenKind): string {
4      switch (kind) {
5          case "char": return "Ein einzelnes Zeichen";
6          case "union": return "Eine Alternative";
7          case "star": return "Ein Kleene-Stern";
8
9          default: {
10             // Compile-Fehler, falls TokenKind in Zukunft erweitert wird,
11             // ohne diesen Switch-Block anzupassen:
12             const exhaustiveCheck: never = kind;
13             return exhaustiveCheck;
14         }
15     }
16 }

```

Listing 41: Vollständige Fallunterscheidung mit `never`

mypy ein leistungsfähiger statischer Checker. Das Problem resultierte vielmehr aus der Diskrepanz zwischen statischer Typprüfung und der hochdynamischen Laufzeitumgebung der Jupyter-Notebooks.

Ein Notebook baut seinen Ausführungszustand interaktiv und sequenziell auf. Die Einbindung von Fremdcode über Magic Commands (wie `%run`) unterläuft die traditionellen, statisch analysierbaren Modulgrenzen. Werkzeuge wie *nb-mypy* versuchen diese Lücke zu schließen, stoßen jedoch bei komplexen, übergreifenden Abhängigkeiten an Systemgrenzen. In der Praxis führte dies häufig dazu, dass Typfehler mit `# type: ignore` unterdrückt werden mussten, was den architektonischen Nutzen des Typsystems stark minderte.

Die Portierung nach TypeScript stellte daher nicht nur einen Syntaxtausch dar, sondern überführte das Projekt in ein strukturell prüfbares Fundament. Die Auflösung der isolierten Notebook-Zellen in ein klassisches Modulsystem mit regulären `import`-Anweisungen ermöglichte es dem TypeScript-Compiler, Datentypen, Schnittstellen und Kontrollflüsse lückenlos und verlässlich zu verifizieren.

3 RecursiveSet: API und fachliche Modellierung

Wie bereits in Kapitel 1 anhand von Listing 1 und Listing 2 zu sehen war, verhalten sich Mengen in Python und TypeScript nicht gleich. Dieses Kapitel erklärt zunächst, worauf diese Unterschiede fachlich und technisch zurückzuführen sind. Anschließend wird gezeigt, wie die Bibliothek `RecursiveSet` diese Unterschiede für die Arbeit mit mathematischen Mengenstrukturen ausgleicht und dadurch eine an Python angelehnte Arbeitsweise in TypeScript ermöglicht.

3.1 Mathematische Mengen und Wohlunterscheidbarkeit

Was ist überhaupt eine Menge im mathematischen Sinn? Georg Cantor beschreibt eine Menge als „Zusammenfassung bestimmter wohlunterschiedener Objekte [...] zu einem Ganzen“ [1, S. 481]. Für die Umsetzung mengenbasierter Algorithmen in einer Programmiersprache ergibt sich daraus unmittelbar eine praktische Frage: Wann gelten zwei Elemente als gleich?

Python und TypeScript beziehungsweise JavaScript beantworten diese Frage unterschiedlich. In Python können unveränderliche, hashbare Werte wie Tupel als Elemente einer Menge verwendet werden. Zwei Tupel gelten dabei als gleich, wenn sie denselben Inhalt haben. In JavaScript werden Objekte wie Arrays in einem `Set` dagegen über ihre Referenzidentität unterschieden. Zwei Arrays mit gleichem Inhalt gelten also nur dann als gleich, wenn es sich um dieselbe Array-Instanz handelt.

Listing 42 zeigt zunächst das Verhalten in Python:

```
1  # Python: Tupel sind unveränderlich und als Set-Elemente geeignet
2  zahlen_menge: set[tuple[int, int]] = set()
3
4  zahlen_menge.add((1, 2))
5  zahlen_menge.add((2, 1))
6
7  print(zahlen_menge)           # {(1, 2), (2, 1)}
8  print((2, 1) in zahlen_menge) # True
```

Listing 42: Mengenverhalten in Python

Die Mitgliedschaftsabfrage funktioniert hier, weil Tupel unveränderlich sind und in Mengen anhand ihrer Gleichheit berücksichtigt werden können.

Anders verhält es sich in TypeScript, wie Listing 43 zeigt. Dieselbe Modellierungsidee führt zu einem anderen Ergebnis:

```

1  type ZahlenTupel = [number, number];
2  const zahlenMenge = new Set<ZahlenTupel>();
3
4  zahlenMenge.add([1, 2]);
5  zahlenMenge.add([2, 1]);
6
7  console.log(zahlenMenge.has([2, 1])); // false

```

Listing 43: Mengenverhalten in TypeScript

Hier schlägt die Abfrage fehl, weil das neu erzeugte Array `[2, 1]` nicht dieselbe Referenz besitzt wie das zuvor eingefügte Array.

Für Algorithmen aus der diskreten Mathematik und theoretischen Informatik ist dieser Unterschied wesentlich. Python erlaubt mit unveränderlichen Werten wie Tupeln eine deutlich natürlichere Modellierung mathematischer Strukturen. In TypeScript ist für inhaltsbasierte Mengen zusammengesetzter Werte in der Regel eine zusätzliche Repräsentation oder eine eigene Datenstruktur erforderlich.

3.2 Die Datenstruktur `RecursiveSet`

Um die Lehrnotebooks funktionsfähig nach TypeScript portieren zu können, muss die Laufzeitumgebung wieder dazu gezwungen werden, Mengen nach ihrem mathematischen Inhalt zu bewerten. Genau hier setzt die Bibliothek `RecursiveSet` an.

3.3 Strukturierte Elemente: Die `Tuple`-Klasse

Neben Mengen stützen sich theoretische Modelle häufig auf Tupel. Ein Tupel ist eine endliche, geordnete Sequenz von Elementen. Im Gegensatz zu Mengen ist die Reihenfolge hier strikt definiert und Duplikate sind erlaubt. Das bekannteste Beispiel ist ein Koordinatenpaar (x, y) in einem zweidimensionalen Raum. Für zwei Punkte $P = (2, 5)$ und $Q = (5, 2)$ gilt mathematisch strikt $P \neq Q$.

In Python ist dieses mathematische Konzept fest in die Sprache integriert. Ein Python-`tuple` ist strukturell vergleichbar und vor allem unveränderlich (immutable). Einmal definiert, kann die Zahl 2 im Punkt P nicht plötzlich zu einer 99 werden.

Versucht man dies in TypeScript durch gewöhnliche Arrays abzubilden, stößt man auf zwei fatale Probleme: Arrays nutzen nicht nur Referenzidentität (wie in Abschnitt 3.1 gezeigt), sondern sind zudem veränderlich (mutable). Ein Algorithmus könnte eine Koordinate nachträglich überschreiben und so die Integrität der gesamten Menge zerstören.

Die Bibliothek löst dies durch die dedizierte `Tuple`-Klasse, die sich exakt wie ein Python-Tupel verhält: Sie ist tiefengleich (Deep Equality) und strikt unveränderlich.

```
1      # Python: Tupel sind nativ tiefengleich und unveränderlich
2      visited_coords: set[tuple[int, int]] = set()
3      visited_coords.add((2, 5))
4
5      # Eine Abfrage mit einem neuen, inhaltlich gleichen Tupel gelingt:
6      print((2, 5) in visited_coords) # True
```

```
1      // TypeScript: Arrays scheitern, Tuple stellt das Python-Verhalten her
2      const visitedCoords = new RecursiveSet<Tuple<number, number>>>();
3      visitedCoords.add(new Tuple(2, 5));
4
5      // Eine Abfrage mit einer neuen Tuple-Instanz gelingt:
6      const searchPoint = new Tuple(2, 5);
7      console.log(visitedCoords.has(searchPoint)); // true
```

Listing 44: Roboter-Koordinaten: Tuple in Python vs. TypeScript

3.4 Assoziative Daten: Die `RecursiveMap`

Sobald Koordinaten oder Zustände korrekt modelliert sind, müssen diesen oft Werte zugeordnet werden. In der Informatik übernimmt dies eine Map (auch Dictionary genannt). Denkt man an ein Schachbrett, so ordnet man einer Koordinate eine Spielfigur zu. Mathematisch ist dies eine Funktion $f(x, y) \rightarrow \text{Wert}$, beispielsweise $f((2, 5)) = \text{„Queen“}$.

Auch hier scheitern TypeScripts native Strukturen. Eine native `Map<number[], string>` würde das Array `[2, 5]` wieder nur als Speicheradresse betrachten. In Python hingegen können Tuple problemlos als Schlüssel in einem Dictionary (Dict) verwendet werden. Als direktes Äquivalent bietet die Bibliothek daher die `RecursiveMap` an:

```

1      # Python: Tupel als Dictionary-Schlüssel funktionieren nativ
2      grid: dict[tuple[int, int], str] = {}
3      grid[(2, 5)] = "Queen"
4
5      print(grid[(2, 5)]) # "Queen"

1      // TypeScript: RecursiveMap nutzt die Wertsemantik für Schlüssel
2      const safeGrid = new RecursiveMap<Tuple<[number, number]>, string>();
3      safeGrid.set(new Tuple(2, 5), "Queen");
4
5      // Der Abruf über eine neue, tiefengleiche Instanz funktioniert:
6      const foundPiece = safeGrid.get(new Tuple(2, 5));
7      console.log(foundPiece); // "Queen"

```

Listing 45: Dictionaries in Python vs. RecursiveMap in TypeScript

3.5 Der Freeze-Contract: Immutabilität sichern

3.6 Von der Mengenkompensation zu expliziten Operationen

3.7 Funktionale Transformationen als Komprehensions-Ersatz

Der weitreichendste sprachliche Unterschied zwischen den Python-Notebooks und der TypeScript-Portierung liegt in der Transformation von Mengen. Python glänzt hier durch sogenannte Mengenkompensationen (Set Comprehensions), die Selektion und Transformation in einer einzigen, mathematisch äußerst prägnanten Zeile vereinen.

Listing 46 zeigt diesen klassischen Python-Ansatz: Aus einer Menge von Zahlen sollen alle ungeraden Zahlen herausgefiltert und das Ergebnis jeweils mit 10 multipliziert werden.

```

1      mixed_numbers = {1, 2, 3, 4, 5}
2
3      # Die Logik verschmilzt direkt mit der Syntax der Sprache
4      odd_tens = { n * 10 for n in mixed_numbers if n % 2 != 0 }
5
6      print(odd_tens) # {10, 30, 50}

```

Listing 46: Python: Mengenkompensation mit Selektion und Transformation

Da TypeScript Syntax nicht durch eine externe Bibliothek um derartige Kon-

strukture erweitert werden kann, muss ein anderes Paradigma genutzt werden: Funktionale Programmierung. Die Bibliothek stellt Methoden bereit, die auf der Menge operieren. Doch wie übergibt man diesen Methoden die individuelle Logik, ohne für jede Kleinigkeit eine eigene benannte Funktion schreiben zu müssen?

Exkurs: Lambda-Funktionen (Arrow Functions)

Die Lösung sind Lambda-Funktionen, in TypeScript „Arrow Functions“ genannt. Eine Arrow Function ist eine anonyme (namenlose) Funktion, die direkt dort definiert wird, wo sie gebraucht wird. Die Syntax lautet `(parameter) => expression`. Der Pfeil `=>` lässt sich am besten als mathematisches „wird abgebildet auf“ lesen. Der Ausdruck `n => n * 10` bedeutet also fachlich exakt: *Nimm das Element n und bilde es auf $n \cdot 10$ ab.*

Mit diesem Werkzeug ausgestattet, können wir die kompakte Python-Komprehension durch die Methode `filterMap` aus `RecursiveSet` nachbauen.

```
1      const mixedNumbers = new RecursiveSet(1, 2, 3, 4, 5);
2
3      const oddTens = mixedNumbers.filterMap(
4          n => n % 2 !== 0,
5          n => n * 10
6      );
7
8      console.log(oddTens.toString()); // {10, 30, 50}
```

Listing 47: TypeScript: Funktionale Pipeline mit `filterMap` und Lambdas

Um diesen Paradigmenwechsel vollständig zu durchdringen, betrachten wir den TypeScript-Code aus Listing 47 Zeile für Zeile:

- **Zeile 3:** Wir rufen die Methode `filterMap` auf unserer Ausgangsmenge auf. Diese Methode iteriert intern über jedes einzelne Element der Menge.
- **Zeile 4:** Dies ist das *Prädikat* (die Filter-Bedingung). Wir übergeben die Lambda-Funktion `n => n % 2 !== 0`. Die Methode wendet diese Funktion auf jedes Element n an. Gibt die Funktion `false` zurück, wird das Element verworfen.
- **Zeile 5:** Dies ist die *Transformation*. Für jedes Element, das den Filter passiert hat, wird nun die zweite Lambda-Funktion `n => n * 10` aufgerufen. Das Ergebnis wird in die neue Zielmenge eingefügt.

Das Ergebnis ist semantisch identisch zum Python-Code, es entsteht jedoch

keine einzige unnötige Zwischenmenge im Speicher, da Selektion und Transformation in einem einzigen Durchlauf evaluiert werden.

3.7.1 Basis-Transformationen: `map`, `filter` und `reduce`

3.7.2 Komplexe Konstruktionen: Verschachtelte Iterationen auflösen mit `flatMap`

Noch deutlicher wird die Stärke der funktionalen Lambda-Konzepte, wenn in Python verschachtelte Schleifen (Nested Comprehensions) verwendet werden. Ein typisches Beispiel aus der theoretischen Informatik ist die Berechnung von Folgezuständen. Für jeden aktuellen Zustand muss hierbei oft eine ganze Teilmenge von neuen Zuständen berechnet und in einer gemeinsamen Menge gesammelt werden.

In Python reiht man dafür einfach mehrere `for`-Klauseln hintereinander, wie Listing 48 zeigt.

```
1 integer_set = {0}
2 natural_numbers = [1, 2, 3]
3
4 # Das äußere 'for' läuft über natural_numbers,
5 # das innere (implizite) fügt +n und -n in die Zielmenge ein.
6 expanded_set = { val for n in natural_numbers for val in (n, -n) }
7
8 # integer_set und expanded_set müssen manuell vereinigt werden
9 integer_set |= expanded_set
10 print(integer_set) # {0, 1, -1, 2, -2, 3, -3}
```

Listing 48: Python: Verschachtelte Mengenkomprehension

Für genau diesen Fall der „Eins-zu-Vielen“-Abbildung (ein Eingabewert generiert eine Menge von Ausgabewerten) stellt `RecursiveSet` die Methode `flatMap` bereit. Sie iteriert über eine Eingabe, führt für jedes Element eine Lambda-Funktion aus, die eine Teilmenge zurückgibt, und „flacht“ all diese Teilmengen direkt in die aufrufende Menge ab.

Listing 49 zeigt die direkte TypeScript-Entsprechung.


```

1      const integerSet = new RecursiveSet<number>(0);
2      const naturalNumbers = [1, 2, 3];
3
4      integerSet.flatMap(
5      naturalNumbers,
6      n => new RecursiveSet<number>(n, -n)
7      );
8
9      console.log(integerSet.toString()); // {0, 1, -1, 2, -2, 3, -3}

```

Listing 49: TypeScript: Direkte Generierung und Vereinigung durch flatMap

Auch hier hilft eine schrittweise Aufschlüsselung der funktionalen Mechanik:

- **Zeile 4–5:** Wir rufen `flatMap` auf dem bereits existierenden `integerSet` auf und übergeben als Quelle `naturalNumbers`.
- **Zeile 6:** Dies ist die Generator-Logik. Die Lambda-Funktion `n => new RecursiveSet(n, -n)` nimmt sich nacheinander jede natürliche Zahl und erschafft eine temporäre Menge, die den positiven und negativen Wert enthält.
- **Die Abflachung (Flattening):** Die Methode `flatMap` nimmt diese zurückgegebene Teilmenge und fügt deren Inhalt direkt (in-place) in das aufrufende `integerSet` ein.

Zusammenfassend lässt sich festhalten: Die TypeScript-Portierung liest sich aufgrund der fehlenden Komprehensions-Syntax grundlegend anders als das Python-Original. Die mathematische Ausdrucksstärke geht jedoch nicht verloren. Sie wird lediglich von der Syntax-Ebene (dem Compiler) auf die Daten-Ebene verlagert, indem die Algorithmen als kompakte Lambda-Funktionen verpackt und an die API übergeben werden.

4 RecursiveSet: Interne Architektur und Implementierung

Während Kapitel 3 die fachliche API und die konzeptionelle Notwendigkeit einer wertsemantischen Mengenstruktur behandelte, fokussiert dieses Kapitel die zugrunde liegende technische Architektur. Da das native JavaScript-`Set` keine strukturelle Gleichheit für komplexe Objekte unterstützt und sich nicht durch einfache Wrapper modifizieren lässt, erfordert `RecursiveSet` eine vollständige Eigenimplementierung als Hash-Tabelle.

Eine Hashtabelle ist im Kern ein Array, bei dem der Speicherort eines Elements direkt aus seinem Wert berechnet wird. Diese direkte Berechnung ermöglicht das eigentliche Hauptziel der Architektur: eine Suchzeit von amortisiert $\mathcal{O}(1)$. Die $\mathcal{O}$ -Notation beschreibt in der Informatik das Laufzeitverhalten eines Algorithmus [2, S. 50f]. $\mathcal{O}(1)$ bedeutet „konstante Zeit“, egal ob die Menge zehn oder zehn Millionen Elemente enthält, das Finden eines Elements dauert im Durchschnitt immer exakt gleich lang. Diese Effizienz, gepaart mit einer schonenden Speicherverwaltung für die Garbage Collection, bildet die zentrale Herausforderung dieses Kapitels.

4.1 Hash-Dispatcher und Gleichheitsprüfung

Die technische Basis für diesen extrem schnellen $\mathcal{O}(1)$ -Zugriff bildet die strikte Trennung von Hash-Berechnung und der tatsächlichen strukturellen Gleichheitsprüfung. Man kann sich den Hash-Wert wie die Postleitzahl und den Straßennamen auf einem Brief vorstellen: Er grenzt das Suchgebiet (den Suchraum) massiv ein. Die Gleichheitsprüfung ist dann der genaue Blick auf das Klingelschild, um sicherzustellen, dass man wirklich beim richtigen Empfänger gelandet ist.

Die Funktion `getHashCode` agiert als Dispatcher (Verteiler). Sie untersucht den Datentyp und wählt die passende Hash-Strategie.

```

1  const HASH_C1 = 0xcc9e2d51;
2  const HASH_C2 = 0x1b873593;
3
4  function getHashCode(val: Value): number {
5      // --- FALL A: ZAHLEN ---
6      if (typeof val === 'number') {
7          // Fast Path: 32-Bit Integers
8          if ((val | 0) === val) {
9              let h = val | 0;
10             h = Math.imul(h ^ (h >>> 16), 0x85ebca6b);
11             h = Math.imul(h ^ (h >>> 13), 0xc2b2ae35);
12             return (h ^ (h >>> 16)) >>> 0;
13         }
14
15         // Slow Path: IEEE 754 Doubles
16         _f64[0] = val;
17         const low = _i32[0];
18         const high = _i32[1];
19         return (Math.imul(low, HASH_C1) ^ Math.imul(high, HASH_C2)) >>> 0;
20     }
21
22     // --- FALL B: STRINGS ---
23     if (typeof val === 'string') {
24         let h = 0x811c9dc5;
25         const len = val.length;
26         for (let i = 0; i < len; i++) {
27             h ^= val.charCodeAt(i);
28             h = Math.imul(h, 0x01000193);
29         }
30         return h >>> 0;
31     }
32
33     // --- FALL C: STRUKTURELLE OBJEKTE ---
34     if (val && typeof val === 'object') {
35         return val.hashCode();
36     }
37
38     return 0;
39 }

```

Listing 50: Der vollständige Hash-Dispatcher der Bibliothek

Dieser Code mag auf den ersten Blick einschüchternd wirken, lässt sich aber sehr logisch in drei Fälle zerlegen:

Fall A: Zahlen (Numbers)

JavaScript besitzt historisch keinen reinen Ganzzahl-Datentyp (Integer), sondern verarbeitet jede `number` unter der Haube als 64-Bit-Fließkommazahl nach dem IEEE-754-Standard [3]. Der Code teilt sich zur Optimierung in zwei Pfade auf:

- **Fast Path (Ganzzahlen):** Die Operation `(val | 0)` schneidet eventuelle Nachkommastellen ab. Ist das Ergebnis identisch mit dem Ursprungswert, haben wir eine saubere Ganzzahl. Würden wir die nackten Zahlen 1, 2 und 3 einfach direkt als Hash-Werte (also als "Parkplatznummern") nutzen, entstünden in der Tabelle sofort Klumpen. Um Kollision (Hashing)s zu vermeiden, wird der Wert stattdessen durch sogenanntes *Avalanche-Mixing* (Bit-Streuung) vermischt. So landen auch dicht beieinanderliegende Zahlen auf völlig unterschiedlichen Plätzen in der Tabelle.
- **Slow Path (Fließkommazahlen):** Handelt es sich um eine Kommazahl (z. B. 3.14), wird sie über einen `ArrayBuffer` in ihre tatsächlichen Nullen und Einsen im Arbeitsspeicher zerlegt. Aus diesen zwei 32-Bit-Hälften (`low` und `high`) wird der Hash berechnet.

In beiden Pfaden tauchen „Magic Numbers“ (wie `0x85ebca6b`) und die Funktion `Math.imul` auf. Das Präfix `0x` kennzeichnet Hexadezimalzahlen. Diese speziellen Werte wurden von Mathematikern so berechnet, dass ihre Bits beim Mischen (`~`) eine optimale Streuung ergeben. `Math.imul` wiederum zwingt die JavaScript-Engine (V8) zu einer echten, hardwarenahen 32-Bit-Ganzzahl-Multiplikation, um ungenaue Rundungsfehler bei Kommazahlen zu vermeiden.

Fall B: Strings (Zeichenketten)

Für Strings verwendet der Dispatcher eine Variante des bekannten *Jenkins One-at-a-Time* Hash-Algorithmus. Dabei wird der String nicht als Ganzes betrachtet, sondern Zeichen für Zeichen in einer `for`-Schleife durchlaufen.

Der numerische Wert jedes Zeichens (`charCodeAt`) wird in den bisherigen Zwischen-Hash eingemischt (`~`) und anschließend multipliziert. Dieses Vorgehen ist essenziell, denn es garantiert, dass nicht nur die Art der Buchstaben, sondern auch ihre *Reihenfolge* den Hash bestimmt. Die Strings `"ab"` und `"ba"` enthalten zwar dieselben Buchstaben, führen durch die schrittweise Einmischung aber korrekterweise zu völlig unterschiedlichen Hash-Werten.

Fall C: Strukturelle Objekte

Handelt es sich bei dem Wert um ein komplexes Objekt (wie etwa unser `Tuple` aus dem vorherigen Kapitel), versucht die Bibliothek nicht, das Objekt mühsam von außen zu analysieren. Sie delegiert die Verantwortung einfach an das Objekt selbst, indem sie die vertraglich zugesicherte Eigenschaft `val.hashCode` abrufen.

Die finale Prüfung mit `areEqual`

Der mühsam berechnete Hash-Wert führt uns nun lediglich zum richtigen „Parkplatz“ in der Hashtabelle. Ob das dort liegende Element wirklich exakt das gesuchte ist, entscheidet abschließend die Funktion `areEqual` :

```
1 function areEqual(a: Value, b: Value): boolean {
2     if (a === b) return true;
3     if (typeof a !== typeof b) return false;
4
5     if (typeof a === 'number' || typeof a === 'string') return a === b;
6
7     if (typeof a === 'object' && a !== null && b !== null) {
8         return a.equals(b);
9     }
10    return false;
11 }
```

Listing 51: Die Gleichheitsfunktion `areEqual`

Auch diese Funktion ist kurz und extrem effizient:

1. **Identitäts-Kurzschluss:** Der einfache `a === b` Vergleich ganz oben fängt identische Speicherreferenzen sofort ab. Das erspart teure Tiefenprüfungen, wenn wir aus Versehen zweimal exakt dasselbe Objekt vergleichen.
2. **Primitive Werte:** Sind beide Werte einfache Zahlen oder Strings, greift der native JavaScript-Vergleich (`a === b`). Wir haben den Hash-Wert vorab nur gebraucht, um sie in der Tabelle zu finden – für den finalen inhaltlichen Beweis reicht das primitive Gleichheitszeichen.
3. **Strukturelle Objekte:** Handelt es sich um komplexe Typen, wird die finale Entscheidung an die vertraglich zugesicherte `equals()` -Methode des jeweiligen Objekts delegiert.

Erst durch das Zusammenspiel dieser beiden Funktionen – `getHashCode` für das schnelle Eingrenzen des Suchraums und `areEqual` für den beweisenden Strukturabgleich – wird die effiziente Wertsemantik der Bibliothek überhaupt erst möglich.

4.2 Speicherlayout: Structure of Arrays (SoA)

[Image comparing Structure of Arrays (SoA) vs Array of Structures (AoS) memory layout]

Ein intuitiver, objektorientierter Entwurf für eine Hash-Map würde sogenannte Bucket-Objekte der Form `{ key, value, hash }` anlegen. Dies nennt man **Array of Structures** (AoS). In der V8-Engine von JavaScript führt dies bei tausenden kleinen Objekten jedoch zu Speicherfragmentierung und Latenzen durch das ständige Aufräumen des Speichers.

`RecursiveMap` (die architektonische Basis für `RecursiveSet`) nutzt stattdessen das **Structure of Arrays** (SoA) Pattern. Statt viele kleine Objekte zu bauen, werden die Eigenschaften in parallelen, einfachen Listen (Arrays) gespeichert. Der Schlüssel an Index 5 gehört exakt zum Wert an Index 5. Dies schont den CPU-Cache.

```
1      class RecursiveMap<K extends Value, V extends Value> {
2          // Dichte Arrays (Nutzdaten liegen direkt nebeneinander)
3          protected readonly _keys: K[] = [];
4          protected readonly _values: V[] = [];
5          protected readonly _hashes: number[] = [];
6
7          // Indextabelle (Suchstruktur für Open Addressing)
8          protected readonly _indices: Uint32Array;
9      }
```

Listing 52: Speicherarchitektur von RecursiveMap

Die Nutzdaten wachsen dicht (ohne Lücken) in den Arrays `_keys`, `_values` und `_hashes`. Die eigentliche Hash-Tabelle ist das `Uint32Array` namens `_indices`. Es enthält keine Objekte, sondern **1-basierte Zeiger** auf die dichten Arrays. Ein Wert von 0 markiert einen leeren Slot. Ein Wert $n > 0$ verweist auf den Index $n - 1$ in den dichten Arrays.

4.3 Kollisionsauflösung via Open Addressing

[Image illustrating Open Addressing with Linear Probing in Hash Tables]

Das Nachschlagen und Einfügen erfolgt über Open Addressing mit Linear Probing. Man kann sich das wie einen Parkplatz vorstellen: Der Hash-Wert berechnet Ihren zugewiesenen Parkplatz. Ist dieser bereits von einem anderen Auto belegt (Kollision), fahren Sie einfach zum nächstmöglichen freien Parkplatz weiter. Standardwerke der Algorithmik zeigen, dass diese Methode bei gut dimensionierten Tabellen hochgradig Cache-effizient ist [vgl. 2, Kap. 11.4].

Der Suchalgorithmus profitiert hier maßgeblich vom SoA-Layout: Bevor die teure Methode `areEqual` aufgerufen wird, vergleicht die Schleife blitzschnell den vorberechneten Hash.

```

1      get(key: K): V | undefined {
2          const h = getHashCode(key);
3          let idx = h & this._mask; // Start-Parkplatz (Bucket) berechnen
4
5          while (true) {
6              const entry = this._indices[idx];
7              if (entry === 0) return undefined; // Slot leer -> Element existiert nicht
8
9              const ptr = entry - 1;
10             // 1. Hash vergleichen (schnell), 2. equals vergleichen (teuer)
11             if (this._hashes[ptr] === h && areEqual(this._keys[ptr], key)) {
12                 return this._values[ptr];
13             }
14             idx = (idx + 1) & this._mask; // Nächster Slot (Linear Probing)
15         }
16     }

```

Listing 53: Lookup-Algorithmus mit linearem Sondieren

4.4 Löschvorgang: Backshift Deletion und Dense-Swap

[Image illustrating Backshift Deletion or Robin Hood Hashing in Hash Tables]

Das Entfernen von Elementen aus Hash-Tabellen mit Open Addressing ist tückisch. Wenn ein Auto den Parkplatz verlässt und wir den Platz einfach leeren, reißt die Suchkette für alle Autos ab, die wegen einer Kollision weiter hinten parken mussten. Die simple Lösung – das Aufstellen eines "GesperrtSchildes (Tombstone (Grabstein)) – führt langfristig zu verschmutzten Tabellen.

Die Bibliothek verwendet stattdessen die elegante Backshift Deletion (eng verwandt mit Robin Hood Hashing). Die entstandene Lücke wird intelligent durch nachfolgende Elemente aufgefüllt, sofern diese dadurch näher an ihren idealen Ursprungs-Hash-Slot heranrücken können [vgl. 2].

```

1      private removeIndex(holeIdx: number): void {
2          let i = (holeIdx + 1) & this._mask;
3          while (this._indices[i] !== 0) {
4              const entry = this._indices[i];
5              const h = this._hashes[entry - 1];
6
7              const idealIdx = h & this._mask;
8              const distToHole = (holeIdx - idealIdx + this._bucketCount) & this._mask;
9              const distToI = (i - idealIdx + this._bucketCount) & this._mask;
10
11              // Element i ist näher an seinem idealen Slot, wenn es in die Lücke rückt
12              if (distToHole < distToI) {
13                  this._indices[holeIdx] = entry; // Lücke füllen
14                  holeIdx = i;                      // Lücke wandert weiter nach hinten
15              }
16              i = (i + 1) & this._mask;
17          }
18          this._indices[holeIdx] = 0; // Lücke ans Ende der Kette verschoben und geleert
19      }

```

Listing 54: Reparatur der Suchkette (Backshifting)

Nachdem die Suchstruktur repariert ist, muss das Element aus den dichten Nutzdaten (`_keys`, `_values`) entfernt werden. Ein `Array.splice()` würde ein teures Verschieben aller nachfolgenden Elemente in $\mathcal{O}(N)$ erzwingen (wie das Zusammenrücken von Büchern im Regal).

Stattdessen nutzt die Implementierung den **Dense-Swap**: Das zu löschende Element wird mit dem *allerletzten* Element im Array überschrieben (das letzte Buch wird einfach in die Lücke gestellt). Anschließend wird das letzte Element mit `pop()` in $\mathcal{O}(1)$ entfernt und der Zeiger in der Indextabelle aktualisiert. Dies garantiert rasend schnelles Löschen ohne Speicher-Neuzuweisung.

4.5 Kapazitätsverwaltung und Hashing-Metadaten

Um die Zugriffszeit von $\mathcal{O}(1)$ aufrechtzuerhalten, darf eine Hash-Tabelle nie komplett voll werden. Der Auslastungsgrad (Load Factor (Auslastungsgrad)) wird daher strikt bei 0.75 (75%) abgeriegelt. Wird dieser überschritten, verdoppelt `ensureCapacity` die Tabellengröße. Die Kapazität bleibt dabei stets eine Potenz von 2 (z. B. 16, 32, 64). Dies ist ein bekannter Trick, um langsame Modulo-Divisionen durch eine extrem schnelle bitweise UND-Operation (`& this._mask`) zu ersetzen [vgl. 2, Kap. 11].

Um Speicherlecks bei häufigen Löschvorgängen zu vermeiden, greift ein **Shrink*-Mechanismus*: Fällt die Auslastung unter 25%, wird die Indextabelle hal-

biert und neu gehasht.

4.6 RecursiveSet: Architektonische Spezialisierung

Das `RecursiveSet` verwendet exakt dieselbe Speicherarchitektur wie die `Map`, eliminiert jedoch das `_keys`-Array, da in einer reinen Menge die gespeicherten Werte zeitgleich ihre eigenen Suchschlüssel sind.

Eine Besonderheit liegt in der `hashCode`-Generierung der Menge selbst. Da Mengen mathematisch ungeordnet sind (die Menge $\{A, B\}$ ist gleich $\{B, A\}$), muss ihr berechneter Hash unabhängig von der Einfügereihenfolge sein. Dies wird durch die kumulative XOR (Exklusives Oder)-Verknüpfung (`^=`) der Element-Hashes im Feld `this._xorHash` erreicht, da XOR kommutativ ist.

Immutabilität und der Freeze-Zyklus

Wie in Abschnitt ?? erläutert, muss eine Menge eingefroren werden, sobald sie in eine andere Struktur eingebettet wird. Technisch wird dies durch den Aufruf von `get hashCode()` erzwungen.

Sobald das Set gehasht wird, setzt es intern `this._isFrozen = true`. Zudem greift ein kaskadierender Freeze: Der Algorithmus iteriert über alle gespeicherten Objekte und ruft deren `hashCode` auf. Dies garantiert, dass auch tief verschachtelte Strukturen iterativ bis in die unterste Ebene versiegelt werden.

4.7 Performance-Optimierung funktionaler Methoden

Die funktionalen Methoden (`map`, `filterMap`, `flatMap`) basieren technisch nicht auf den langsameren JavaScript-Iteratoren. Um den Overhead von Generator-Funktionen zu vermeiden, iterieren diese Methoden imperativ über klassische `for`-Schleifen direkt über das dichte `_values`-Array.

Zudem wird bei Transformationen eine optimistische Prä-Allokation durchgeführt: `result.ensureCapacity(this.size)`. Unter der Annahme, dass die Zielmenge ähnlich groß wird wie die Ursprungsmenge, wird der benötigte Speicher direkt am Stück reserviert, was teure Tabellenvergrößerungen während der Schleife eliminiert.

4.8 Fazit der internen Architektur

Die interne Umsetzung demonstriert, dass strukturelle Wertsemantik in TypeScript nicht durch einfache Typ-Deklarationen erreicht werden kann. Es bedarf einer vollständigen, auf die Hardware-Architektur abgestimmten Datenstruktur.

Durch das SoA-Paradigma, die intelligente Trennung von Hash-Wegweiser und inhaltlicher Gleichheitsprüfung sowie Algorithmen wie Backshift-Deletion und

Dense-Swapping wird `RecursiveSet` zu einer hochperformanten Infrastruktur. Sie bietet die formale Strenge mathematischer Mengen und liefert gleichzeitig die Ausführungsgeschwindigkeit, die Studierende benötigen, um komplexe Automaten-Algorithmen ressourcenschonend zu implementieren.

5 Vom Python-Werkzeug *PLY* zu *Lezer*

In den ursprünglichen Python-Implementierungen der Lehrveranstaltung werden zentrale Schritte der Sprachverarbeitung mit der Bibliothek *PLY* (Python Lex-Yacc) umgesetzt. *PLY* ist eine Parsergenerator-Bibliothek für Python, die sich an den klassischen Werkzeugen *Lex* und *Yacc* orientiert. Diese Werkzeuge werden häufig in Lehrumgebungen eingesetzt, um grundlegende Konzepte der Compilertechnik wie Tokenisierung und Parsing praktisch umzusetzen. Die Bibliothek wurde von David Beazley entwickelt und ist frei verfügbar [4].

Die grundlegende Architektur eines solchen Sprachprozessors besteht aus zwei zentralen Phasen: der lexikalischen Analyse und der syntaktischen Analyse. Zunächst zerlegt der Lexer den Eingabetext in eine Folge elementarer Token, anschließend analysiert der Parser diese Tokenfolge anhand einer kontextfreien Grammatik und konstruiert daraus eine syntaktische Struktur, etwa einen abstrakten Syntaxbaum (AST) [5, S.5ff].

PLY bildet diese beiden Komponenten explizit ab:

- **Lexikalische Analyse (Lexer):** Token werden über reguläre Ausdrücke definiert. Der Lexer erkennt daraus elementare Spracheinheiten wie Zahlen, Operatoren oder Schlüsselwörter.
- **Syntaxanalyse (Parser):** Die Grammatik der Sprache wird als Menge von Produktionsregeln formuliert. *PLY* generiert daraus einen Parser, der auf LR-Parsing-Techniken basiert. Diese Parsingverfahren lesen die Eingabe von links nach rechts und konstruieren eine rechtsableitende Analyse, wodurch auch komplexe Programmiersprachen effizient verarbeitet werden können.

Ein charakteristisches Merkmal von *PLY* ist, dass Token-Definitionen und Grammatikregeln direkt im Python-Code definiert werden. Produktionsregeln werden dabei als Funktionen formuliert, deren Docstrings die jeweilige Grammatikregel enthalten. Diese enge Integration erleichtert die prototypische Entwicklung von Parsern im Lehrkontext.

Während *PLY* somit eine klassische Compilerarchitektur innerhalb der Python-Umgebung realisiert, verfolgt das TypeScript-Ökosystem einen anderen Ansatz. Für editornähe Anwendungen und webbasierte Entwicklungsumgebungen wird häufig das Parser-Framework *Lezer* eingesetzt.

Lezer verfolgt das Ziel, syntaktische Analysen effizient und inkrementell durchzuführen. Im Gegensatz zu traditionellen Parsergeneratoren wird der Parser so konstruiert, dass er Änderungen am Quelltext lokal verarbeiten kann, ohne den gesamten Syntaxbaum neu berechnen zu müssen [6].

```

1 tokens = ('NUMBER', 'PLUS')
2
3 t_PLUS = r'\+'
4
5 def t_NUMBER(t):
6     r'\d+'
7     t.value = int(t.value)
8     return t

```

Listing 55: Ply-Grammatik in Python

Die Grammatik wird hierbei nicht mehr als Python-Code formuliert, sondern als deklarative Grammatikbeschreibung in einer eigenen Syntax definiert:

```

1 @top Expression { Add }
2
3 Add {
4     Expression "+" Expression
5 }

```

Listing 56: Lezer-Grammatik in TypeScript

Aus dieser Beschreibung generiert *Lezer* einen Parser, der direkt im TypeScript-Ökosystem verwendet werden kann.

Der zentrale Unterschied zwischen beiden Ansätzen liegt somit weniger in der zugrundeliegenden formalen Theorie. Beide basieren auf kontextfreien Grammatiken und klassischen Parsingverfahren, sie unterscheiden sich vielmehr in der Zielumgebung. Während *PLY* primär für klassische Compilerprototypen und Lehrzwecke in Python konzipiert wurde, ist *Lezer* auf moderne Entwicklungswerkzeuge und inkrementelle Analyse von Programmtexten ausgelegt.

Im Kontext dieser Arbeit dient *PLY* daher als Referenzimplementierung der ursprünglichen Python-Notebooks, während *Lezer* die Grundlage für die Umsetzung der gleichen Sprachverarbeitungsschritte im TypeScript-Ökosystem bildet.

5.1 Grundlagen von Lezer

Lezer ist ein Parser-Framework, das im Umfeld des CodeMirror-Projekts entwickelt wurde und speziell auf den Einsatz in modernen Entwicklungswerkzeugen ausgelegt ist [6]. Im Gegensatz zu vielen klassischen Parsergeneratoren steht bei *Lezer* nicht die einmalige Analyse eines vollständigen Programms im Vordergrund, sondern die effiziente Verarbeitung sich kontinuierlich ändernder Programmtexte.

Ein zentrales Ziel von *Lezer* besteht darin, die syntaktische Struktur eines Textes auch während der Bearbeitung im Editor schnell verfügbar zu machen. Funktionen wie Syntax-Highlighting, automatische Einrückung oder Fehlermarkierungen benötigen eine konsistente syntaktische Analyse des aktuellen Dokumentzustands. Da sich der Quelltext während der Bearbeitung ständig verändert, muss der Parser in der Lage sein, diese Änderungen effizient zu verarbeiten.

Inkrementelles Parsing

Eine zentrale Eigenschaft von *Lezer* ist das sogenannte *inkrementelle Parsing*. Dabei wird der Syntaxbaum eines Dokuments nicht bei jeder Änderung vollständig neu berechnet. Stattdessen werden nur jene Teile des Baums aktualisiert, die tatsächlich von der Änderung betroffen sind.

Dieser Ansatz reduziert den Rechenaufwand erheblich und ermöglicht eine nahezu verzögerungsfreie Aktualisierung der Syntaxstruktur im Editor. Besonders bei größeren Dokumenten oder komplexeren Grammatiken stellt dies einen entscheidenden Vorteil gegenüber klassischen Parserarchitekturen dar [7].

Baumrepräsentation der Syntax

Das Ergebnis der Analyse durch *Lezer* ist eine baumartige Repräsentation der syntaktischen Struktur des Eingabetextes. Diese Struktur enthält sämtliche syntaktischen Elemente, die durch die Grammatik definiert wurden.

Lezer erzeugt dabei zunächst einen sogenannten *Concrete Syntax Tree* (CST). Im Gegensatz zu einem abstrakten Syntaxbaum bleiben hier auch Zwischensymbole und strukturelle Details der Grammatik erhalten. Diese vollständige Darstellung der syntaktischen Struktur ist besonders für editornahe Funktionen relevant, da sie eine präzise Lokalisierung einzelner Sprachelemente ermöglicht.

Integration in das JavaScript- und TypeScript-Ökosystem

Ein weiterer wichtiger Aspekt von *Lezer* ist seine Integration in das JavaScript- und TypeScript-Ökosystem. Der generierte Parser kann direkt in Webanwendungen oder Entwicklungswerkzeuge (wie z.B. in Jupyter-Notebook) eingebunden werden.

Im Kontext dieser Arbeit wird *Lezer* verwendet, um die syntaktische Analyse der ursprünglichen Python-Lehrbeispiele in eine TypeScript-basierte Implementierung zu übertragen. Die Grammatik der Sprache bildet dabei die formale Grundlage für die Generierung des Parsers, während TypeScript für die Weiterverarbeitung der analysierten Strukturen eingesetzt wird.

Der nächste Abschnitt beschreibt daher die konkrete Definition der Grammatik, die als Grundlage für die spätere Parsergenerierung dient.

5.2 Grammatikdefinition in Lezer

Die syntaktische Struktur der zu analysierenden Sprache wird in *Lezer* durch eine deklarative Grammatikbeschreibung festgelegt. Diese Grammatik bildet die Grundlage für die spätere Generierung des Parsers und definiert, welche syntaktischen Strukturen innerhalb der Sprache zulässig sind.

Im Rahmen dieser Arbeit wurde die Grammatik direkt innerhalb der verwendeten Jupyter-Notebooks entwickelt. Dieses Vorgehen orientiert sich an der Struktur der ursprünglichen Lehrmaterialien, die ebenfalls notebookbasiert aufgebaut sind. Dadurch konnte die Grammatik schrittweise erweitert und unmittelbar anhand von Beispielausdrücken getestet werden.

Ein wesentliches Merkmal der vorliegenden Implementierung besteht darin, dass die Grammatik nicht in einer separaten Datei abgelegt wurde, sondern als mehrzeiliger String direkt im TypeScript-Code des Notebooks definiert ist. Die Grammatik wird somit als Datenobjekt innerhalb des Programms gehalten und kann ohne Zwischenschritt unmittelbar an die Parsergenerierung übergeben werden. Dieses Vorgehen vereinfacht den experimentellen Umgang mit der Grammatik, da Änderungen direkt an der Stringdefinition vorgenommen und sofort getestet werden können.

Struktur einer Lezer-Grammatik

Eine *Lezer*-Grammatik besteht aus mehreren Komponenten, die gemeinsam die syntaktische Struktur der Sprache beschreiben. Dazu gehören insbesondere:

- die Definition eines Startsymbols,
- Regeln zur Beschreibung komplexer syntaktischer Strukturen,
- sowie Token-Definitionen für elementare Sprachelemente.

Das folgende Beispiel zeigt eine vereinfachte Grammatikdefinition für arithmetische Ausdrücke, wie sie im Notebook als String festgehalten wird:

Die Variable `grammarDefinition` enthält die vollständige Grammatikbeschreibung als Zeichenkette. Durch diese Repräsentation bleibt die Grammatik eng mit dem restlichen Notebook-Code verknüpft. Insbesondere kann sie direkt an die Funktion zur Parsererzeugung übergeben werden, ohne dass ein externer Lade- oder Build-Schritt erforderlich ist.

Das Schlüsselwort `@top` definiert das Startsymbol der Grammatik. Dieses Symbol bildet den Einstiegspunkt der syntaktischen Analyse. Die Regel `Add` beschreibt eine mögliche Struktur eines Ausdrucks: Ein Ausdruck kann entweder aus zwei Teilausdrücken bestehen, die durch einen Additionsoperator verbunden sind, oder aus einem numerischen Wert.

```

1  const grammarDefinition = `
2      @top Expression { Add }
3
4      Add {
5          Expression "+" Expression |
6          Number
7      }
8
9      @tokens {
10         Number { ${[0-9]}+ }
11     }
12 `;

```

Listing 57: Umsetzung Lezer-Grammatik in TypeScript

Die Definition der Token erfolgt im Block `@tokens`. Hier werden reguläre Ausdrücke verwendet, um elementare Spracheinheiten zu erkennen. Diese Token bilden die Grundlage für die syntaktische Analyse, da sie die kleinsten Einheiten darstellen, aus denen komplexere Strukturen aufgebaut werden.

Iterative Entwicklung der Grammatik

Die Entwicklung der Grammatik erfolgte iterativ innerhalb der Notebook-Umgebung. Nach jeder Anpassung der Stringdefinition konnte der Parser neu generiert und direkt auf Beispielausdrücke angewendet werden. Dieses Vorgehen erleichtert insbesondere die schrittweise Erweiterung der Grammatik, da syntaktische Fehler oder Mehrdeutigkeiten frühzeitig erkannt werden können.

Darüber hinaus unterstützt die Notebook-Umgebung eine experimentelle Arbeitsweise, bei der einzelne Sprachkonstrukte isoliert getestet werden können. Dies ist insbesondere im Lehrkontext hilfreich, da die Auswirkungen einzelner Grammatikregeln unmittelbar sichtbar werden.

Die so definierte Grammatik bildet schließlich die Grundlage für die Generierung des Parsers, die im folgenden Abschnitt beschrieben wird.

5.3 Generierung des Parsers

Nachdem die Grammatik definiert wurde, kann daraus ein ausführbarer Parser erzeugt werden. In klassischen Projekten erfolgt dieser Schritt häufig über ein separates Generatorwerkzeug im Build-Prozess. Im Rahmen dieser Arbeit wurde die Parsergenerierung jedoch direkt innerhalb der Notebook-Umgebung durchgeführt.

Hierzu ist insbesondere das Paket `@lezer/generator` relevant. Nach der offiziellen Dokumentation übernimmt dieses Paket die Erzeugung der Parse-Tabellen

aus einer Grammatikbeschreibung. Üblicherweise wird es als Generatorwerkzeug im Build-Prozess eingesetzt; es kann jedoch auch direkt als JavaScript-Funktion verwendet werden [6]. Genau dieses Vorgehen wurde in der vorliegenden Arbeit genutzt.

Die grundlegende Erstellung des Parsers erfolgt über die Funktion `buildParser`, die aus `@lezer/generator` importiert wird. Diese Funktion nimmt die zuvor definierte Grammatik als Eingabe und erzeugt daraus eine Parserinstanz:

```
1 import { buildParser } from "@lezer/generator";  
2  
3 const parser = buildParser(grammarDefinition);
```

Listing 58: Erstellung des Parsers in TypeScript

Die Variable `grammarDefinition` enthält hierbei die deklarative Beschreibung der Grammatik. Auf Grundlage dieser Definition erzeugt *Lezer* die internen Parsingtabellen und Datenstrukturen, die für die syntaktische Analyse benötigt werden. Der so erzeugte Parser basiert zur Laufzeit auf dem *Lezer-LR*-System, während die erzeugten Syntaxbäume auf den gemeinsamen Baumstrukturen des *Lezer*-Ökosystems beruhen [6].

Parserinstanz

Das Ergebnis des Generierungsprozesses ist ein Parserobjekt, das anschließend zur Analyse von Eingabetexten verwendet werden kann. Der Parser verarbeitet einen Eingabestring und erzeugt daraus eine Baumstruktur, welche die syntaktische Struktur des Ausdrucks repräsentiert.

Diese Struktur ermöglicht es, einzelne Sprachelemente gezielt zu identifizieren und weiterzuverarbeiten. Insbesondere können nachgelagerte Verarbeitungsschritte auf bestimmte Knoten des Syntaxbaums zugreifen, um beispielsweise Ausdrücke auszuwerten oder Transformationen durchzuführen.

Integration in den Notebook-Workflow

Die Generierung des Parsers innerhalb des Notebooks bietet mehrere Vorteile für den Entwicklungsprozess. Änderungen an der Grammatik können unmittelbar getestet werden, da der Parser nach jeder Anpassung neu erzeugt werden kann. Dadurch entsteht ein iterativer Entwicklungsprozess, bei dem Grammatikdefinition und Parserverhalten eng miteinander verknüpft sind.

Dieses Vorgehen entspricht der didaktischen Struktur der ursprünglichen Lehrmaterialien, in denen Sprachkonstrukte schrittweise eingeführt und unmittelbar anhand von Beispielen überprüft werden.

Der generierte Parser bildet schließlich die Grundlage für die syntaktische Analyse innerhalb der TypeScript-Implementierung. Im folgenden Abschnitt wird beschrieben, wie der Parser in die eigentliche Programmlogik integriert wird und wie die erzeugten Syntaxbäume weiterverarbeitet werden.

5.4 Integration in die TypeScript-Implementierung

Die Generierung des Parsers stellt im Projekt keinen isolierten Endpunkt dar, sondern ist unmittelbar in die weitere TypeScript-Implementierung eingebettet. Der mit `buildParser(grammarDefinition)` erzeugte Parser wird direkt zur Analyse von Eingabestrings verwendet und bildet damit die Eingangsstufe für die weitere Verarbeitung der Sprachelemente.

Im konkreten Projekt greifen dabei mehrere Pakete des *Lezer*-Ökosystems zusammen. Das Paket `@lezer/generator` wird zur Erzeugung des Parsers aus der Grammatikdefinition verwendet. Das Paket `@lezer/common` stellt mit `TreeCursor` eine zentrale Schnittstelle für die Traversierung der erzeugten Baumstruktur bereit. Zusätzlich wird `@lezer/lr` für den LR-basierten Parser-Typ verwendet, sodass die Parserinstanz auch auf Typebene explizit beschrieben werden kann [6].

Parsing von Eingabetexten

Nach der Erzeugung der Parserinstanz kann ein Eingabetext über die Methode `parse` analysiert werden. Diese Methode ist nicht projektspezifisch implementiert, sondern Bestandteil der durch *Lezer* erzeugten Parserinstanz. Im Projekt wird sie über den Ausdruck `parser.parse(input)` aufgerufen. Das Ergebnis ist zunächst keine fachliche Datenstruktur der Anwendung, sondern eine vom Parser erzeugte syntaktische Baumrepräsentation. Diese Struktur enthält die durch die Grammatik erkannten Knoten und dient als unmittelbares Resultat der *Lezer*-Analyse.

Im Notebook wird dieser Schritt direkt mit der anschließenden Weiterverarbeitung verbunden. Ein typisches Muster besteht darin, den Eingabetext zunächst durch den generierten Parser analysieren zu lassen, anschließend einen Cursor auf dem erzeugten Baum zu erzeugen und diesen Baum danach in eine projektspezifische Datenstruktur zu überführen:

Dieses Beispiel verdeutlicht, dass im Projekt zwischen der von *Lezer* bereitgestellten Parsingfunktion und der eigenen Anwendungslogik unterschieden wird. Während `parser.parse(...)` die konkrete Syntax analysiert und einen Syntaxbaum erzeugt, übernimmt die Funktion `parseStmnt` die Kapselung dieses Schritts sowie die anschließende Transformation in eine typisierte interne Repräsentation.

```

1 import { buildParser } from "@lezer/generator";
2 import { TreeCursor } from "@lezer/common";
3
4 function parseStmtnt(input: string): Stmtnt {
5     const tree = parser.parse(input);
6     const ast = cstToAST(tree.cursor(), input, config);
7     if (ast instanceof AssignNode || ast instanceof ExprStmtNode) {
8         return ast;
9     }
10    throw new Error(`Expected a statement, got: ${ast.constructor.name}`);
11 }

```

Listing 59: ParseStmtnt Funktion in TypeScript

Transformation vom CST zum projektspezifischen AST

Für die eigentliche Programmlogik genügt die konkrete Syntaxstruktur allein nicht. Benötigt wird vielmehr eine Datenstruktur, die auf die Anforderungen der weiteren Verarbeitung abgestimmt ist. Aus diesem Grund wird der durch *Lezer* erzeugte Baum in einen projektspezifischen abstrakten Syntaxbaum (AST) überführt.

Im Projekt erfolgt diese Transformation über die Funktion `cstToAST`. Diese verarbeitet den vom Parser erzeugten Baum rekursiv anhand eines `TreeCursor`. Für jeden Knoten wird geprüft, ob eine zugehörige Transformationsregel definiert ist. Ist dies der Fall, wird aus den Kindknoten ein passender AST-Knoten konstruiert. Knoten, die lediglich syntaktische Zwischenstufen darstellen, können dabei ausgelassen oder automatisch reduziert werden. Die entsprechende Transformationslogik ist im Projekt generisch über `ParserConfig` und `TransformRule` organisiert, sodass die Abbildung vom konkreten Syntaxbaum auf den projektspezifischen AST regelbasiert erfolgt.

Dieses Vorgehen trennt die parsernahe Syntaxrepräsentation klar von der semantisch relevanten internen Modellierung. Die von *Lezer* erzeugte Struktur bleibt damit auf die Analyse des Eingabetexts beschränkt, während die eigentliche Programmlogik ausschließlich mit den definierten TypeScript-Typen und Klassen arbeitet.

Typsicherheit an der Schnittstelle

Ein weiterer Vorteil dieser Integration liegt in der klaren Typisierung der Schnittstelle zwischen Parser und Anwendungscode. Die Funktion `parseStmtnt` kapselt nicht nur den Aufruf des generierten Parsers, sondern überprüft zusätzlich, ob das Resultat tatsächlich die erwartete Form besitzt. Dadurch wird verhindert, dass nachgelagerte Verarbeitungsschritte mit ungeeigneten oder unerwarteten Knotenstrukturen arbeiten.

Gerade im Kontext dieser Arbeit ist dieser Schritt von Bedeutung, da die Portierung nach TypeScript nicht nur die syntaktische Analyse reproduzieren soll, sondern zugleich die Vorteile statischer Typisierung nutzbar macht. Die Integration des Parsers endet daher nicht beim Erzeugen eines Syntaxbaums, sondern umfasst auch dessen kontrollierte Überführung in wohldefinierte, typisierte Datenstrukturen.

Bedeutung für die Gesamtarchitektur

Die Einbettung des *Lezer*-Parsers in die TypeScript-Implementierung zeigt, dass der Parser im Projekt als Teil einer mehrstufigen Verarbeitungskette verstanden wird. Auf die formale Grammatikdefinition folgt die Parsererzeugung, anschließend die Analyse konkreter Eingaben durch den generierten Parser und schließlich die Transformation in eine interne Repräsentation, auf der Auswertung, Analyse oder weitere Übersetzungsschritte aufbauen können.

Damit übernimmt *Lezer* im Gesamtsystem die Aufgabe, die textuelle Eingabe in eine formal strukturierte Zwischenform zu überführen. Die fachliche Bedeutung der erkannten Konstrukte entsteht jedoch erst durch die nachgelagerte Abbildung auf die projektspezifischen AST-Klassen. Genau diese Trennung zwischen syntaktischer Analyse und semantischer Weiterverarbeitung macht die Integration in die TypeScript-Implementierung besonders anschlussfähig und wartbar.

6 Literaturverzeichnis

- [1] Georg Cantor. „Beiträge zur Begründung der transfiniten Mengenlehre“. In: *Mathematische Annalen* 46 (1895), S. 481–512. URL: <http://eudml.org/doc/157768>.
- [2] Thomas H. Cormen u. a. *Introduction to Algorithms*. 4. Aufl. MIT Press, 2022. ISBN: 978-0262046305.
- [3] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019 (Revision of IEEE 754-2008). IEEE, Juli 2019. URL: https://www-users.cse.umn.edu/~vinals/tspot_files/phys4041/2020/IEEE%20Standard%20754-2019.pdf.
- [4] David Beazley. *PLY (Python Lex-Yacc)*. 2026. URL: <https://www.dabeaz.com/ply/> (besucht am 08.02.2026).
- [5] Alfred V. Aho u. a. *Compilers: Principles, Techniques, and Tools*. 2. Aufl. Addison-Wesley, 2006. ISBN: 978-0321486813.
- [6] *Lezer Documentation*. CodeMirror / Lezer Project. 2026. URL: <https://lezer.codemirror.net/docs/> (besucht am 08.02.2026).
- [7] Marijn Haverbeke. *Lezer*. 3. Sep. 2019. URL: <https://marijnhaverbeke.nl/blog/lezer.html> (besucht am 08.02.2026).

A Anhang

A.1 Verzeichnis der portierten Lehrnotebooks

Die folgende Tabelle listet alle Jupyter-Notebooks auf, die im Rahmen dieser Arbeit selektiert und von Python nach TypeScript portiert wurden. Die Gliederung folgt der Kapitelstruktur der Originalvorlesung „Theoretische Informatik III“.

Tabelle 1: Übersicht der übersetzten Notebooks im Repository

Original-Kapitel	Notebook-Dateiname
Chapter 03	01-Ply-Scanning-Example.ipynb 02-Regex-Tutorial.ipynb 03-Exam-Evaluation.ipynb 04-Html2Text.ipynb
Chapter 04 & 05	01-NFA-2-DFA.ipynb 02-Test-NFA-2-DFA.ipynb 03-Regex-2-NFA.ipynb 04-Test-Regex-2-NFA.ipynb 05-DFA-2-RegExp.ipynb 06-Test-DFA-2-RegExp.ipynb 07-Minimize.ipynb 08-Test-Minimize.ipynb 09-Equivalence.ipynb 10-Test-Equivalence.ipynb
Chapter 06	01-Top-Down-Parser.ipynb 02-EBNF-Parser.ipynb
Chapter 07	01-Calculator.ipynb 02-Differentiator.ipynb 03-Interpreter.ipynb
Chapter 10	01-Conflicts.ipynb 02-Conflicts-Resolved.ipynb

Quelle: Eigene Darstellung.